

Rapport TP2 ANN201

Simulation numérique de l'écoulement d'un fluide avec des éléments finis
 P^2 - P^1

Tom Cuel

December 6, 2024



Contents

1	Introduction	3
2	Éléments finis P^2 pour le problème de Poisson avec condition de Neumann	4
2.1	Formulation variationnelle	4
2.2	Discrétisation	5
2.3	Géométrie et maillage	6
2.4	Calcul des matrices élémentaires par passage à l'élément fini de référence	7
2.5	Assemblage des matrices globales	13
2.6	Calcul du second membre	13
2.7	Validation de la méthode	14
3	Éléments finis P^2 pour le problème de Poisson avec condition de Dirichlet	20
3.1	Conditions de Dirichlet homogènes	20
3.1.1	Formulation variationnelle discrète	20
3.1.2	Résolution informatique	21
3.1.3	Validation	22
3.2	Conditions de Dirichlet non homogènes	25
3.2.1	Prise en compte de la non homogénéité du problème dans la pseudo-élimination	25
3.2.2	Validation	26
4	Équations de Stokes	29
4.1	Obtention de la formulation variationnelle	29
4.2	Nouveau Maillage	32
4.3	Discrétisation	32
4.4	Calcul des matrices élémentaires	33
4.5	Assemblage des matrices	34
4.6	Second membre et pseudo-élimination	36
4.7	Validation avec un exemple	36
4.7.1	Calcul de $\vec{g}$	36
4.7.2	Solution exacte, vérification	37
4.7.3	Premier résultats	37
4.7.4	Analyse des résultats	42
4.8	Écoulement d'un fluide dans un canal avec une marche descendante	43
4.8.1	Modification du maillage	43
4.8.2	Modification des routines de pseudo-élimination et de second membre	45
4.8.3	Validation	46
4.8.4	Influence de la viscosité	48

1 Introduction

On s'intéresse dans ce TP à l'écoulement d'un fluide visqueux incompressible et stationnaire dans un canal bidimensionnel, en régime à faible nombre de Reynolds. Ce problème est modélisé par les *équations de Stokes*

$$\begin{cases} -\nu\Delta\vec{u} + \nabla p = f & \text{dans } \Omega \\ \operatorname{div}(\vec{u}) = 0 & \text{dans } \Omega \end{cases} \quad (1)$$

qui correspondent à une simplification des équations de Navier-Stokes dans lesquelles on a négligé le phénomène de convection par rapport au phénomène de diffusion. On aura ici $u \in H^1(\Omega) \times H^1(\Omega)$ la vitesse du fluide et $p \in H^1(\Omega)$ la pression.

Ces équations présentent de multiples difficultés que nous allons aborder progressivement. En effet, leur discrétisation nécessite l'utilisation d'éléments finis P^2 . Avant d'employer ce type d'éléments finis pour les équations de Stokes, nous allons d'abord les mettre en œuvre sur des problèmes de Poisson.

2 Éléments finis P^2 pour le problème de Poisson avec condition de Neumann

Soit Ω un ouvert borné de $\mathbb{R}^2$ à frontière polygonale et $f \in L^2(\Omega)$. On rappelle le problème du TP1 : trouver $u \in H^1(\Omega)$ tel que

$$\begin{cases} u - \Delta u = f & \text{dans } \Omega \\ \frac{\partial u}{\partial n} = 0 & \text{sur } \partial\Omega \end{cases} \quad (2)$$

2.1 Formulation variationnelle

Retrouvons la formulation variationnelle discrète du problème comme fait dans le TP1 :

On multiplie par une fonction test $v \in H^1(\Omega)$ et on intègre sur Ω :

$$\int_{\Omega} uv \, d\Omega - \int_{\Omega} \Delta u v \, d\Omega = \int_{\Omega} f v \, d\Omega.$$

Cependant, $u \in H^1(\Omega)$ et $\Delta u = u + f \in L^2(\Omega)$, donc $u \in H^1(\Omega, \Delta)$, on peut donc supposer $u \in H^2(\Omega)$ et utiliser la formule de Green :

$$\forall v \in H^1(\Omega), \int_{\Omega} (uv + \nabla u \cdot \nabla v) \, d\Omega + \int_{\partial\Omega} (\nabla u \cdot \vec{n} v) \, d\Gamma = \int_{\Omega} f v \, d\Omega$$

Or, on a $\nabla u \cdot \vec{n} = 0$ sur $\partial\Omega$, donc :

$$\boxed{\text{Trouver } u \in H^1(\Omega) \text{ telle que } \forall v \in H^1(\Omega), \int_{\Omega} (uv + \nabla u \cdot \nabla v) \, d\Omega = \int_{\Omega} f v \, d\Omega} \quad (3)$$

Montrons maintenant l'équivalence entre la formulation variationnelle et le problème de Poisson. Prenons dans un premier temps φ dans $D(\Omega)$, on a :

$$\int_{\Omega} \nabla u \cdot \nabla \varphi \, d\Omega = \int_{\Omega} (f - u) \varphi \, d\Omega$$

On a :

$$\int_{\Omega} \nabla u \cdot \nabla \varphi \, d\Omega = \langle \nabla u, \nabla \varphi \rangle \quad \text{car } \nabla u \in L^2(\Omega) \subset \mathcal{D}'(\Omega)$$

$$\text{Donc, } \langle \nabla u, \nabla \varphi \rangle = \sum_{i=1}^d \left\langle \frac{\partial u}{\partial x_i}, \frac{\partial \varphi}{\partial x_i} \right\rangle = - \sum_{i=1}^d \left\langle \frac{\partial^2 u}{\partial x_i^2}, \varphi \right\rangle = - \langle \Delta u, \varphi \rangle$$

par définition de la dérivation au sens des distributions

Et comme $f - u \in L^2(\Omega)$, on a :

$$\int_{\Omega} (f - u) \varphi \, d\Omega = \langle f - u, \varphi \rangle.$$

On en déduit que

$$-\Delta u = f - u \quad \text{dans } \mathcal{D}'(\Omega).$$

Comme le second membre est dans $L^2(\Omega)$, l'égalité est vraie dans $L^2(\Omega)$ presque partout. Nous en déduisons de plus que $u \in H^1(\Omega, \Delta)$.

Pour les conditions aux limites, on a en choisissant $v \in H^1(\Omega)$ et on peut supposer $u \in H^2(\Omega)$ comme $u \in H^1(\Omega, \Delta)$, on peut appliquer la formule de Green :

$$\int_{\Omega} (uv - \Delta u v) \, d\Omega + \int_{\partial\Omega} (\nabla u \cdot \vec{n} v) \, d\Gamma = \int_{\Omega} f v \, d\Omega$$

donc :

$$= \int_{\Omega} (u - f - \Delta u) v \, d\Omega + \int_{\partial\Omega} (\nabla u \cdot \vec{n} v) \, d\Gamma = 0$$

On applique le résultat dans $\mathcal{D}'(\Omega)$ et on a, comme l'intégrale dans Ω est nulle :

$$\int_{\partial\Omega} (\nabla u \cdot \vec{n} v) d\Gamma = 0$$

On peut écrire :

$$\nabla u \cdot \vec{n} \in \text{Im}(\gamma_0)^\perp$$

Or, $\text{Im}(\gamma_0)$ est dense dans $L^2(\partial\Omega)$, donc $\nabla u \cdot \vec{n} = 0$ presque partout sur $\partial\Omega$, d'où l'équivalence du problème.

Pour pouvoir utiliser le théorème de Lax-Milgram, on définit :

$$\begin{aligned} a : H^1(\Omega) \times H^1(\Omega) &\longrightarrow \mathbb{R} \\ (u, v) &\longmapsto \int_{\Omega} (uv + \nabla u \cdot \nabla v) d\Omega \end{aligned}$$

Et on définit :

$$\begin{aligned} l : H^1(\Omega) &\longrightarrow \mathbb{R} \\ v &\longmapsto \int_{\Omega} f v d\Omega \end{aligned}$$

On peut donc réécrire le problème comme suit :

$$\boxed{\text{Trouver } u \in H^1(\Omega) \text{ telle que } \forall v \in H^1(\Omega), a(u, v) = l(v)} \quad (4)$$

Pour montrer que le problème est bien posé, on va utiliser le théorème de Lax-Milgram. Vérifions ses hypothèses :

Premièrement, on a $H^1(\Omega)$ est un espace de Hilbert pour la norme $\|\cdot\|_{H^1(\Omega)}$
 a est clairement bilinéaire et continue (*preuve ci dessous par Chauchy-Schwarz*).

$$\begin{aligned} |a(u, v)| &\leq \|u\|_{L^2(\Omega)} \cdot \|v\|_{L^2(\Omega)} + \|\nabla u\|_{L^2(\Omega)} \cdot \|\nabla v\|_{L^2(\Omega)} \\ &\leq 2\|u\|_{H^1(\Omega)} \cdot \|v\|_{H^1(\Omega)} \text{ (2 pour majorer sans se poser de questions)} \end{aligned}$$

De plus, a est coercive :

$$\begin{aligned} a(u, u) &= \int_{\Omega} (u^2 + \|\nabla u\|^2) d\Omega \\ &= \|u\|_{H^1(\Omega)}^2 \\ &\geq \|u\|_{H^1(\Omega)}^2 \end{aligned}$$

l est clairement linéaire et continue (*preuve ci dessous par Chauchy-Schwarz*).

$$\begin{aligned} |l(v)| &\leq \|f\|_{L^2(\Omega)} \cdot \|v\|_{L^2(\Omega)} \\ &\leq \|f\|_{L^2(\Omega)} \cdot \|v\|_{H^1(\Omega)}. \end{aligned}$$

On a donc bien vérifié les hypothèses du théorème de Lax-Milgram, $\boxed{\text{le problème est donc bien posé}}.$

2.2 Discrétisation

Soit $\mathcal{T}_h$ une triangulation du domaine Ω et $(T_\ell)_{\ell=1,L}$ les triangles de $\mathcal{T}_h$. Nous considérons V_h l'approximation de $H^1(\Omega)$ par des éléments finis P^2 associés à la triangulation $\mathcal{T}_h$:

$$V_h = \{v_h \in \mathcal{C}^0(\overline{\Omega}), v_h|_{T_\ell} \in \mathbb{P}^2 \forall \ell = 1, \dots, L\},$$

où $\mathbb{P}^2$ est l'espace des polynômes de degré au plus 2. Les fonctions de base sont associées aux sommets des triangles et aux milieux des arêtes (voir Figure 1). On note donc $(M_I)_{I=1,N}$ les noeuds du maillage (qui incluent les sommets et les milieux des arêtes) et $(w_I)_{I=1,N}$ la base de V_h définie par $w_I(M_J) = \delta_{IJ}$, $1 \leq I, J \leq N$.

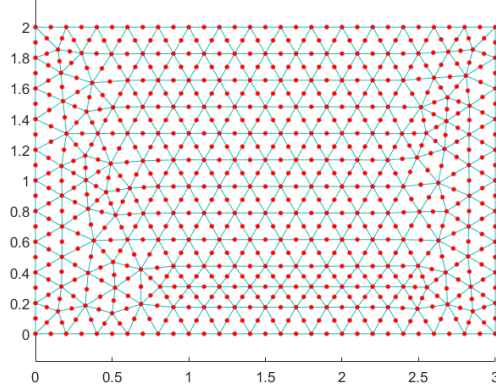


Figure 1: Exemple de maillage avec les noeuds (sommets et milieux des triangles) associés à des éléments finis P^2

On effectue la discrétisation :

$$\boxed{\text{Trouver } u_h \in V_h \text{ tel que } \forall v_h \in V_h, a(u_h, v_h) = l(v_h)} \quad (5)$$

Qui s'écrit aussi :

$$\boxed{\text{Trouver } u_h \in V_h \text{ tel que } \forall v_h \in V_h, \int_{\Omega} (u_h v_h + \nabla u_h \cdot \nabla v_h) d\Omega = \int_{\Omega} f v_h d\Omega} \quad (6)$$

Par construction de la solution approchée (*On décompose u_h sur la base des w_I*), on a :

$$\forall (x, y) \in \overline{\Omega}^2, u_h(x, y) = \sum_{I=1}^N u_h(M_I) w_I(x, y)$$

On injecte cette expression dans l'équation variationnelle discrète et on obtient :

$$\sum_{I=1}^N u_h(M_I) \int_{\Omega} (w_I v_h + \nabla w_I \cdot \nabla v_h) d\Omega = \int_{\Omega} f v_h d\Omega$$

On choisit en particulier $v_h = w_J$ pour $J \in \{1, 2, \dots, N\}$, on alors :

$$\sum_{I=1}^N \left(\int_{\Omega} w_I w_J + \nabla w_I \cdot \nabla w_J d\Omega \right) u_h(M_I) = \int_{\Omega} f w_J d\Omega$$

La nouvelle formulation du problème est donc :

$$\boxed{\begin{aligned} &\text{Trouver } \vec{U} = \begin{pmatrix} U_h(M_1) \\ U_h(M_2) \\ \vdots \\ U_h(M_N) \end{pmatrix} \text{ tel que } (\mathbb{M} + \mathbb{K}) \vec{U} = \vec{L} \\ &\text{où } \forall (I, J) \in \{1, 2, \dots, N\}^2, \mathbb{M}_{IJ} = \int_{\Omega} w_I w_J d\Omega, \mathbb{K}_{IJ} = \int_{\Omega} \nabla w_I \cdot \nabla w_J d\Omega, \text{ et } \vec{L}_I = \int_{\Omega} f w_I d\Omega \end{aligned}} \quad (7)$$

On peut alors effectuer les calculs inverses pour trouver l'équivalence comme fait précédemment, en raisonnant progressivement sur les espaces.

2.3 Géométrie et maillage

On a encore une fois les différentes structures de données lorsque l'on ouvre un maillage, avec ici quelques modifications qui prennent en compte la présence de points au milieu des arêtes :

- Nbpt : nombre de sommets et de milieux d'arêtes (entier)
- Nbtri : nombre de triangles (entier)
- Coorneu : coordonnées (x, y) des sommets et des milieux des arêtes (matrice réelle Nbpt x 2)
- Refneu : reference des sommets et des milieux des arêtes (vecteur entier Nbpt x 1)
- Numtri : liste de triangles (3 numéros de sommets, 3 numéros de milieux - matrice entière de taille Nbtri x 6)
- Reftri : reference des triangles (matrice entière Nbtri x 1)
- Numaretes(Nbaretes,2) = numero des 3 noeuds (*2 sommets + milieu d'arête*) de chaque arête de la frontière
- Nbaretes : Nombre d'aretes sur la frontiere
- Refaretes(Nbaretes,1) = référence de chaque arête de la frontière

On peut lire les différentes infos sur le maillage et afficher les maillages à l'aide des fonctions `lecture_msh_ordre2.m` et `affichemaillage_ordre2.m`, comme dans la figure suivante :

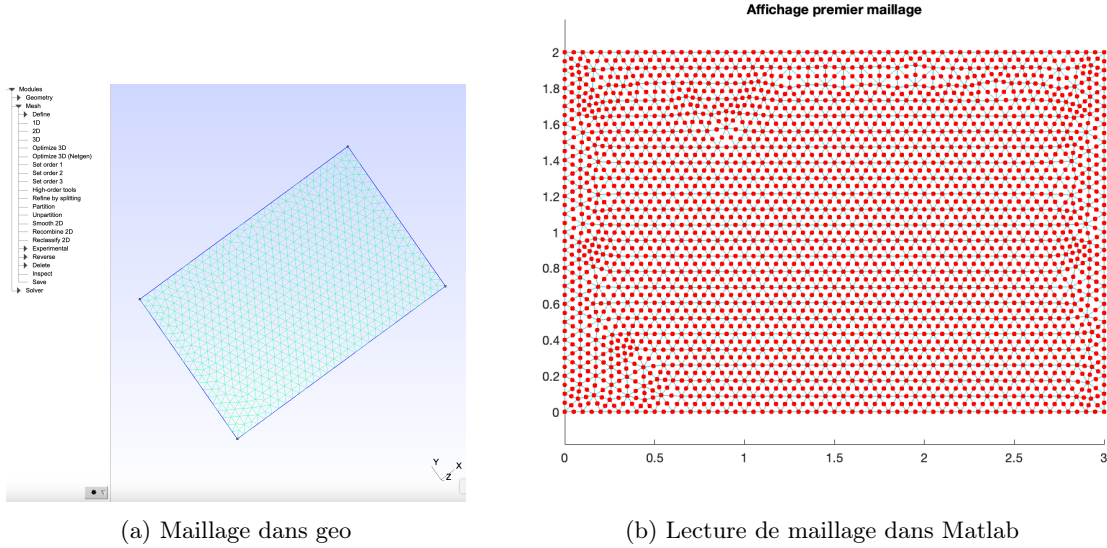


Figure 2: Exemple personnel de maillage 2D d'ordre 2, avec des éléments finis P^2

2.4 Calcul des matrices élémentaires par passage à l'élément fini de référence

Soit T_ℓ un triangle de sommets $M_1 = (x_1, y_1)$, $M_2 = (x_2, y_2)$, $M_3 = (x_3, y_3)$ et de milieux $M_4 = (x_4, y_4)$, $M_5 = (x_5, y_5)$, $M_6 = (x_6, y_6)$. Soient $\lambda_1, \lambda_2, \lambda_3$ les coordonnées barycentriques du triangle. Les fonctions de base locales sont les suivantes :

$w_1 = \lambda_1(2\lambda_1 - 1),$	$w_4 = 4\lambda_1\lambda_2,$
$w_2 = \lambda_2(2\lambda_2 - 1),$	$w_5 = 4\lambda_2\lambda_3,$
$w_3 = \lambda_3(2\lambda_3 - 1),$	$w_6 = 4\lambda_3\lambda_1.$

Voici le code pour ces fonctions de base locales :

```

% q : pour savoir la fonction que l'on doit considérer
% (S_x, S_y) = le point en lequel on doit évaluer
% (x1,y1), (x2,y2), (x3,y3) : les coordonnées des trois sommets du triangle
% (pas forcément de référence ici)
function [w_value] = w(q, Sx, Sy, x1, y1, x2, y2, x3, y3)

    w_value = 0;
    if q == 1
        w_value = lambda(1, Sx, Sy, x1, y1, x2, y2, x3, y3)*(2*lambda(1, Sx, Sy, x1, y1, x2, y2, x3, y3)-1);
    elseif q == 2
        w_value = lambda(2, Sx, Sy, x1, y1, x2, y2, x3, y3)*(2*lambda(2, Sx, Sy, x1, y1, x2, y2, x3, y3)-1);
    elseif q == 3
        w_value = lambda(3, Sx, Sy, x1, y1, x2, y2, x3, y3)*(2*lambda(3, Sx, Sy, x1, y1, x2, y2, x3, y3)-1);
    elseif q == 4
        w_value = 4*lambda(1, Sx, Sy, x1, y1, x2, y2, x3, y3)*lambda(2, Sx, Sy, x1, y1, x2, y2, x3, y3);
    elseif q == 5
        w_value = 4*lambda(2, Sx, Sy, x1, y1, x2, y2, x3, y3)*lambda(3, Sx, Sy, x1, y1, x2, y2, x3, y3);
    elseif q == 6
        w_value = 4*lambda(3, Sx, Sy, x1, y1, x2, y2, x3, y3)*lambda(1, Sx, Sy, x1, y1, x2, y2, x3, y3);
    end
end

```

Figure 3: Code pour les fonctions de base locales

Rappelons tout d'abord l'expression des coordonnées barycentriques λ_1, λ_2 et λ_3 . En un point (x, y) du triangle, celles-ci sont données par les formules suivantes :

$$\begin{aligned}\lambda_1(x, y) &= \frac{1}{D} (y_{23}(x - x_3) - x_{23}(y - y_3)) \\ \lambda_2(x, y) &= \frac{1}{D} (y_{31}(x - x_1) - x_{31}(y - y_1)) \\ \lambda_3(x, y) &= \frac{1}{D} (y_{12}(x - x_2) - x_{12}(y - y_2))\end{aligned}$$

où $x_{ij} = x_i - x_j$, $y_{ij} = y_i - y_j$ pour i et j différents dans $\{1, 2, 3\}$, et $D = x_{23}y_{31} - x_{31}y_{23}$. Notons que D est égal, au signe près, à deux fois la surface du triangle.

Voici le code pour les fonctions barycentriques :

```

% p : pour savoir la fonction que l'on doit considérer
% (S_x, S_y) = le point en lequel on doit évaluer
% (x1,y1), (x2,y2), (x3,y3) : les coordonnées des trois sommets du triangle
function [lambda_value] = lambda(p, Sx, Sy, x1, y1, x2, y2, x3, y3)

    lambda_value = 0;
    D = (x2-x1)*(y3-y1) - (x3-x1)*(y2-y1);
    if p == 1
        lambda_value = ((y2-y3)*(Sx-x3) - (x2-x3)*(Sy-y3))/D;
    elseif p == 2
        lambda_value = ((y3-y1)*(Sx-x1) - (x3-x1)*(Sy-y1))/D;
    elseif p == 3
        lambda_value = ((y1-y2)*(Sx-x2) - (x1-x2)*(Sy-y2))/D;
    end
end

```

Figure 4: Code pour les fonctions barycentriques

Pour calculer les matrices élémentaires associées à T_ℓ , nous allons utiliser la méthode du *passage à l'élément fini de référence*. On se ramène tout d'abord au triangle de référence $\hat{T}$, c'est-à-dire le triangle de sommets $\hat{M}_1 = (0, 0)$, $\hat{M}_2 = (1, 0)$ et $\hat{M}_3 = (0, 1)$.

On cherche la transformation $\mathcal{F}_\ell : \mathbb{R}^2 \rightarrow \mathbb{R}^2$

$$\forall \hat{M} \in \mathbb{R}^2, \mathcal{F}_\ell(\hat{M}) = \mathcal{B}_\ell \hat{M} + S_\ell, \mathcal{B}_\ell \in \mathcal{M}_2(\mathbb{R}), S_\ell \in \mathbb{R}^2$$

qui permet de passer du triangle de référence $\hat{T}$ au triangle T_ℓ :

$$\forall i = 1, 2, 3, \mathcal{F}_\ell(\hat{M}_i) = M_i.$$

Écrivons $\mathcal{B}_\ell$ et S_ℓ en fonction de leurs coefficients b_{ij} et s_i :

$$\mathcal{B}_\ell = \begin{pmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{pmatrix}, \quad S_\ell = \begin{pmatrix} s_1 \\ s_2 \end{pmatrix}$$

On a alors, en évaluant $\mathcal{F}_\ell$ en $\hat{M}_i$, $\forall i = 1, 2, 3$:

$$\left\{ \begin{array}{l} \begin{pmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{pmatrix} \cdot \begin{pmatrix} 0 \\ 0 \end{pmatrix} + \begin{pmatrix} s_1 \\ s_2 \end{pmatrix} = \begin{pmatrix} x_1 \\ y_1 \end{pmatrix} \\ \begin{pmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{pmatrix} \cdot \begin{pmatrix} 1 \\ 0 \end{pmatrix} + \begin{pmatrix} s_1 \\ s_2 \end{pmatrix} = \begin{pmatrix} x_2 \\ y_2 \end{pmatrix} \\ \begin{pmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{pmatrix} \cdot \begin{pmatrix} 0 \\ 1 \end{pmatrix} + \begin{pmatrix} s_1 \\ s_2 \end{pmatrix} = \begin{pmatrix} x_3 \\ y_3 \end{pmatrix} \end{array} \right\} \iff \left\{ \begin{array}{l} s_1 = x_1 \\ s_2 = y_1 \\ b_{11} = x_2 - x_1 \\ b_{12} = x_3 - x_1 \\ b_{21} = y_2 - y_1 \\ b_{22} = y_3 - y_1 \end{array} \right.$$

On peut donc écrire la transformation $\mathcal{F}_\ell$ explicitement :

$$\mathcal{F}_\ell : \begin{pmatrix} \hat{x} \\ \hat{y} \end{pmatrix} \mapsto \begin{pmatrix} x_1 + (\hat{x} - 1)(x_2 - x_1) + \hat{y}(x_3 - x_1) \\ y_1 + (\hat{x} - 1)(y_2 - y_1) + \hat{y}(y_3 - y_1) \end{pmatrix}$$

Ou en conservant la forme précédente mais avec les expressions explicites de $\mathcal{B}_\ell$ et S_ℓ :

$$\mathcal{F}_\ell : \begin{array}{ccc} \mathbb{R}^2 & \longrightarrow & \mathbb{R}^2 \\ \hat{M} & \longmapsto & \mathcal{B}_\ell \hat{M} + S_\ell \end{array} \text{ où } \mathcal{B}_\ell = \begin{pmatrix} x_2 - x_1 & x_3 - x_1 \\ y_2 - y_1 & y_3 - y_1 \end{pmatrix} \text{ et } S_\ell = \begin{pmatrix} x_1 \\ y_1 \end{pmatrix}$$

Les intégrales de masse (pour la matrice $\mathbb{M}$) sur T_ℓ peuvent alors s'écrire $\forall (I, J) \in \{1, 2, \dots, 6\}^2$:

$$\boxed{\int_{T_\ell} w_I(M) w_J(M) d\Omega = \int_{\hat{T}} \hat{w}_I(\hat{M}) \hat{w}_J(\hat{M}) |\det \mathcal{B}_\ell| d\hat{\Omega}} \quad (8)$$

et les intégrales de rigidité (pour la matrice $\mathbb{K}$) :

$$\boxed{\int_{T_\ell} \nabla w_I(M) \cdot \nabla w_J(M) d\Omega = \int_{\hat{T}} [(\mathcal{B}_\ell^T)^{-1} \hat{\nabla} \hat{w}_I(\hat{M})] \cdot [(\mathcal{B}_\ell^T)^{-1} \hat{\nabla} \hat{w}_J(\hat{M})] |\det \mathcal{B}_\ell| d\hat{\Omega}} \quad (9)$$

F_ℓ permet de passer du triangle de référence $\hat{T}$ au triangle T_ℓ : $\forall i = 1, 2, 3$, $\mathcal{F}_\ell(\hat{M}_i) = M_i$.

On note également pour $i \in N_3$,

$$\hat{w}_I : M \mapsto w(M) = w(F_\ell(\hat{M}))$$

Les intégrales du second membre des équations pourraient être calculées analytiquement, mais, pour les éléments finis P^2 , il s'agit de calculs laborieux. Pour les calculer, nous allons plutôt faire appel à une *formule de quadrature*. De façon générale, une formule de quadrature à N_Q points est définie comme suit : pour G une fonction continue par morceaux de $\hat{T}$, on fait l'approximation

$$\boxed{\int_{\hat{T}} G d\hat{\Omega} \simeq \sum_{q=1}^{N_Q} \hat{c}^q G(\hat{S}^q)} \quad (10)$$

où $\hat{S}^q$ sont des points de quadrature dans $\hat{T}$ et $\hat{c}^q$ des poids positifs associés aux points de quadrature. On dit qu'une formule de quadrature est d'ordre k si elle approche de façon exacte les intégrales des polynômes de degré k . Pour évaluer les intégrales, nous allons donc respectivement utiliser des formules de quadrature d'ordre 4 et 2. Pour la matrice de masse, on utilisera la formule de quadrature à 6 points de Gauss-Legendre qui est d'ordre 4 et qui est définie sur le triangle de référence $\hat{T}$ par

$$\begin{aligned} \hat{S}^1 &= (0.0915762135098, 0.0915762135098), \quad \hat{c}^1 = 0.05497587183 \\ \hat{S}^2 &= (0.8168475729805, 0.0915762135098), \quad \hat{c}^2 = 0.05497587183 \\ \hat{S}^3 &= (0.0915762135098, 0.8168475729805), \quad \hat{c}^3 = 0.05497587183 \\ \hat{S}^4 &= (0.1081030181681, 0.4459484909160), \quad \hat{c}^4 = 0.1116907948 \\ \hat{S}^5 &= (0.4459484909160, 0.1081030181681), \quad \hat{c}^5 = 0.1116907948 \\ \hat{S}^6 &= (0.4459484909160, 0.4459484909160), \quad \hat{c}^6 = 0.1116907948 \end{aligned} \quad (11)$$

Pour la matrice de rigidité, on utilisera la formule de quadrature à 3 points de Gauss-Legendre qui est d'ordre 2 et qui est définie sur le triangle de référence $\hat{T}$ par :

$$\hat{S}^1 = \left(\frac{1}{6}, \frac{1}{6}\right), \quad \hat{S}^2 = \left(\frac{2}{3}, \frac{1}{6}\right), \quad \hat{S}^3 = \left(\frac{1}{6}, \frac{2}{3}\right), \quad \hat{c}^1 = \hat{c}^2 = \hat{c}^3 = \frac{1}{6} \quad (12)$$

Maintenant, calculons les coefficients des matrices élémentaires $\mathbb{M}_\ell^e$ et $\mathbb{K}_\ell^e$ pour chaque triangle T_ℓ de la triangulation $\mathcal{T}_h$. Les matrices élémentaires $\mathbb{M}_\ell^e$ et $\mathbb{K}_\ell^e$ sont tout d'abord dans $\mathbb{M}_6(\mathbb{R})$. Concentrons-nous sur le calcul de $\mathbb{M}_\ell^e$ pour un triangle T_ℓ donné, on traitera ensuite le cas de $\mathbb{K}_\ell^e$.

On a, en utilisant les formules de quadrature, pour $\mathbb{M}_\ell^e$, $\forall(I, J) \in \{1, 2, \dots, 6\}^2$:

$$G(\hat{S}^q) = \hat{w}_I(\hat{S}^q) \hat{w}_J(\hat{S}^q) |\det \mathcal{B}_\ell|$$

On peut alors écrire $\forall(I, J) \in \{1, 2, \dots, 6\}^2$:

$$\boxed{\mathbb{M}_{\ell, IJ}^e \simeq \sum_{q=1}^6 \hat{w}_I(\hat{S}^q) \hat{w}_J(\hat{S}^q) |\det \mathcal{B}_\ell| \hat{c}^q}$$

On a donc, en utilisant les formules précédentes, pour $\hat{w}_I$ avec les $\hat{\lambda}_i$ et l'expression du déterminant de $\mathcal{B}_\ell$:

$$|\det \mathcal{B}_\ell| = |(x_2 - x_1)(y_3 - y_1) - (x_3 - x_1)(y_2 - y_1)| = |D|$$

On peut détailler l'expression du coefficient $\mathbb{M}_{\ell, 11}^e$ afin de se donner une idée de la méthode de calcul :

$$\begin{aligned} \mathbb{M}_{\ell, 11}^e &\simeq \sum_{q=1}^6 \hat{w}_1(\hat{S}^q) \hat{w}_1(\hat{S}^q) |D| \hat{c}^q \\ &= \sum_{q=1}^6 \left(\hat{\lambda}_1(\hat{S}^q) (2\hat{\lambda}_1(\hat{S}^q) - 1) \right)^2 |D| \hat{c}^q \\ &= \sum_{q=1}^6 \left(\frac{y_{23}(\hat{S}_1^q - \hat{x}_3) - x_{23}(\hat{S}_2^q - \hat{y}_3)}{x_{23}\hat{y}_{31} - x_{31}\hat{y}_{23}} \left(2 \frac{y_{23}(\hat{S}_1^q - \hat{x}_3) - x_{23}(\hat{S}_2^q - \hat{y}_3)}{x_{23}\hat{y}_{31} - x_{31}\hat{y}_{23}} - 1 \right) \right)^2 \\ &\quad \times |(x_2 - x_1)(y_3 - y_1) - (x_3 - x_1)(y_2 - y_1)| \hat{c}^q. \end{aligned}$$

Où on se ramène au triangle de référence $\hat{T}$, c'est-à-dire le triangle de sommets $\hat{M}_1 = (\hat{x}_1, \hat{y}_1)$, $\hat{M}_2 = (\hat{x}_2, \hat{y}_2)$ et $\hat{M}_3 = (\hat{x}_3, \hat{y}_3)$.

On ne va pas détailler toutes les formules, une seule est déjà assez lourde alors que c'est la plus réduite possible, comme on est sur un coefficient diagonal. L'implémentation de la routine Matlab pour le calcul des matrices élémentaires sera plus parlante directement, plutôt que d'enchaîner les calculs à la main. Il faut bien faire attention à l'ordre des calculs et à l'évaluation des différentes valeurs.

```

% calcul de la matrice de masse
% -----
% Initialisation
Mel = zeros(6,6);

% Points et poids de quadrature
S_hat = [0.0915762135098, 0.0915762135098;
          0.8168475729805, 0.0915762135098;
          0.0915762135098, 0.8168475729805;
          0.1081030181681, 0.4459484909160;
          0.4459484909160, 0.1081030181681;
          0.4459484909160, 0.4459484909160];
poids = [0.05497587183, 0.05497587183, 0.05497587183, 0.1116907948, 0.1116907948, 0.1116907948];

% Det B_l = D donc on a seulement besoin de D
D = (x2-x1)*(y3-y1) - (x3-x1)*(y2-y1);

% On se place sur le triangle de référence :
x1 = 0;
y1 = 0;
x2 = 1;
y2 = 0;
x3 = 0;
y3 = 1;

for i=1:6
    for j=1:6
        G=0;
        for k=1:6
            G = G + poids(k) * abs(D) * w(i, S_hat(k,1), S_hat(k,2), x1, y1, x2, y2, x3, y3) * w(j, S_hat(k,1), S_hat(k,2), x1, y1, x2, y2, x3, y3);
        end
        Mel(i,j) = G;
    end
end

```

Figure 5: Code calcul de matrice de masse élémentaire

On a, en utilisant les formules de quadrature, pour $\mathbb{K}_\ell^e, \forall (I, J) \in \{1, 2, \dots, 6\}^2$:

$$G(\hat{S}^q) = \left((\mathcal{B}_\ell)^{-1} \cdot \hat{\nabla} \hat{w}_I(\hat{S}^q) \right) \cdot \left((\mathcal{B}_\ell)^{-1} \cdot \hat{\nabla} \hat{w}_J(\hat{S}^q) \right) |\det \mathcal{B}_\ell|$$

Écrivons une fois la formule générale pour $\mathbb{K}_{\ell,IJ}^e, \forall (I, J) \in \{1, 2, \dots, 6\}^2$:

$$\mathbb{K}_{\ell,IJ}^e \simeq \sum_{q=1}^3 \left((\mathcal{B}_\ell^T)^{-1} \hat{\nabla} \hat{w}_I(\hat{S}^q) \right) \cdot \left((\mathcal{B}_\ell^T)^{-1} \hat{\nabla} \hat{w}_J(\hat{S}^q) \right) |\det \mathcal{B}_\ell| \hat{c}^q$$

La tâche est ici encore plus complexe à effectuer à la main, il faudrait inverser la matrice $\mathcal{B}_\ell$ et calculer les gradients des fonctions de base. On va donc directement passer à l'implémentation de la routine Matlab pour le calcul des matrices élémentaires, en calculant les coefficients de la même manière que pour $\mathbb{M}_\ell^e$.

On peut tout de même écrire les gradient des $\hat{w}_I$ en fonction des $\hat{\lambda}_i$:

$$\begin{aligned}
\hat{\nabla} \hat{w}_1 &= (4\hat{\lambda}_1 - 1) \nabla \hat{\lambda}_1 \\
\hat{\nabla} \hat{w}_2 &= (4\hat{\lambda}_2 - 1) \nabla \hat{\lambda}_2 \\
\hat{\nabla} \hat{w}_3 &= (4\hat{\lambda}_3 - 1) \nabla \hat{\lambda}_3 \\
\hat{\nabla} \hat{w}_4 &= 4(\nabla \hat{\lambda}_1 \hat{\lambda}_2 + \hat{\lambda}_1 \nabla \hat{\lambda}_2) \\
\hat{\nabla} \hat{w}_5 &= 4(\nabla \hat{\lambda}_2 \hat{\lambda}_3 + \hat{\lambda}_2 \nabla \hat{\lambda}_3) \\
\hat{\nabla} \hat{w}_6 &= 4(\nabla \hat{\lambda}_3 \hat{\lambda}_1 + \hat{\lambda}_3 \nabla \hat{\lambda}_1)
\end{aligned}$$

On connaît les expressions des gradients des λ_i et des $\nabla \lambda_i$ en fonction des x_i et y_i , comme dans le TP1 :

$$\begin{aligned}
\nabla \hat{\lambda}_1 &= \frac{1}{\hat{D}} \begin{pmatrix} \hat{y}_2 - \hat{y}_3 \\ \hat{x}_3 - \hat{x}_2 \end{pmatrix} \\
\nabla \hat{\lambda}_2 &= \frac{1}{\hat{D}} \begin{pmatrix} \hat{y}_3 - \hat{y}_1 \\ \hat{x}_1 - \hat{x}_3 \end{pmatrix} \\
\nabla \hat{\lambda}_3 &= \frac{1}{\hat{D}} \begin{pmatrix} \hat{y}_1 - \hat{y}_2 \\ \hat{x}_2 - \hat{x}_1 \end{pmatrix}
\end{aligned}$$

On peut coder les gradients des fonctions de base locales :

```

% q : pour savoir la fonction que l'on doit considérer
% (S_x, S_y) = le point en lequel on doit évaluer
% (x1,y1), (x2,y2), (x3,y3) : les coordonnées des trois sommets du triangle
% (pas forcément de référence ici)
function [w_vecteur] = grad_w(q, Sx, Sy, x1, y1, x2, y2, x3, y3)

    D = (x2-x1)*(y3-y1) - (x3-x1)*(y2-y1);
    grad_lambda_1 = [(y2 - y3)/D ; (x3 - x2)/D];
    grad_lambda_2 = [(y3 - y1)/D ; (x1 - x3)/D];
    grad_lambda_3 = [(y1 - y2)/D ; (x2 - x1)/D];
    w_vecteur = [0 ; 0];
    if q == 1
        w_vecteur = (4*lambda(1, Sx, Sy, x1, y1, x2, y2, x3, y3)-1)*grad_lambda_1;
    elseif q == 2
        w_vecteur = (4*lambda(2, Sx, Sy, x1, y1, x2, y2, x3, y3)-1)*grad_lambda_2;
    elseif q == 3
        w_vecteur = (4*lambda(3, Sx, Sy, x1, y1, x2, y2, x3, y3)-1)*grad_lambda_3;
    elseif q == 4
        w_vecteur = 4*(lambda(2, Sx, Sy, x1, y1, x2, y2, x3, y3)*grad_lambda_1 + lambda(1, Sx, Sy, x1, y1, x2, y2, x3, y3)*grad_lambda_2);
    elseif q == 5
        w_vecteur = 4*(lambda(3, Sx, Sy, x1, y1, x2, y2, x3, y3)*grad_lambda_2 + lambda(2, Sx, Sy, x1, y1, x2, y2, x3, y3)*grad_lambda_3);
    elseif q == 6
        w_vecteur = 4*(lambda(1, Sx, Sy, x1, y1, x2, y2, x3, y3)*grad_lambda_3 + lambda(3, Sx, Sy, x1, y1, x2, y2, x3, y3)*grad_lambda_1);
    end
end

```

Figure 6: Code pour les fonctions de base locales

Il faut ensuite tout assembler et bien évaluer correctement les différents valeurs. On a le code suivant pour le calcul de la matrice de rigidité élémentaire :

```

% Initialisation
Kel = zeros(6,6);

% Points et poids de quadrature
S_hat = [1/6, 1/6;
         2/3, 1/6;
         1/6, 2/3];
poids = 1/6;

% Matrice B_l inversée et transposée
Bl = [x2 - x1, x3 - x1;
      y2 - y1, y3 - y1];
Bl_invtranspose = inv(Bl');
% Det Bl = D donc on a seulement besoin de D
D = (x2-x1)*(y3-y1) - (x3-x1)*(y2-y1);

% On effectue la transformation des points pour passer de w à w^
% On se place sur le triangle de référence : ce sont les points obtenus par la transformation F_l
x1 = 0;
y1 = 0;
x2 = 1;
y2 = 0;
x3 = 0;
y3 = 1;

for i=1:6
    for j=1:6
        G=0;
        for k=1:3
            G = G + poids * abs(D) * dot(Bl_invtranspose*grad_w(i, S_hat(k,1), S_hat(k,2), x1, y1, x2, y2, x3, y3), Bl_invtranspose*grad_w(j, S_hat(k,1), S_hat(k,2), x1, y1, x2, y2, x3, y3));
        end
        Kel(i,j) = G;
    end
end

```

Figure 7: Code calcul de matrice de rigidité élémentaire

Comme mentionné précédemment, on ne va pas détailler les tous les calculs à la main, mais on peut calculer juste $\mathbb{M}_{11}^e$:
Sur le triangle de référence, on a :

$$\begin{pmatrix} \hat{x}_1 \\ \hat{y}_1 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \quad \begin{pmatrix} \hat{x}_2 \\ \hat{y}_2 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad \begin{pmatrix} \hat{x}_3 \\ \hat{y}_3 \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

On a alors pour $\hat{\lambda}_1$:

$$\begin{aligned} \hat{\lambda}_1(\hat{x}, \hat{y}) &= \frac{1}{D} (\hat{y}_{23}(\hat{x} - \hat{x}_3) - \hat{x}_{23}(\hat{y} - \hat{y}_3)) \\ &= \frac{1}{1} ((0 - 1) \times (\hat{x} - 0) - (1 - 0) \times (\hat{y} - 1)) \\ &= 1 - \hat{x} - \hat{y} \end{aligned}$$

On a alors pour $\hat{w}_1$:

$$\begin{aligned}\hat{w}_1(\hat{x}, \hat{y}) &= \hat{\lambda}_1(\hat{x}, \hat{y}) \times (2\hat{\lambda}_1(\hat{x}, \hat{y}) - 1) \\ &= (1 - \hat{x} - \hat{y}) \times (2(1 - \hat{x} - \hat{y}) - 1) \\ &= (1 - \hat{x} - \hat{y}) \times (2 - 2\hat{x} - 2\hat{y} - 1) \\ &= (1 - \hat{x} - \hat{y}) \times (1 - 2\hat{x} - 2\hat{y}) \\ &= 1 - 3\hat{x} - 3\hat{y} + 4\hat{x}\hat{y} + 2\hat{x}^2 + 2\hat{y}^2\end{aligned}$$

On aurait pu faire comme cela pour les autres fonctions de base, mais cela aurait été très long et fastidieux (*Cela revenait à faire $2 \times (3 + 6)$ calculs (avec les gradients)*). On a donc préféré directement coder les calculs dans Matlab.

2.5 Assemblage des matrices globales

Pour assembler les matrices élémentaires, on utilise l'algorithme d'assemblage que j'ai implémenté dans Matlab. Ici, i va de 1 à 6 car on ne considère plus seulement les sommets du triangle mais aussi les milieux des arêtes. On a donc 6 fonctions de base.

```
% boucle sur les triangles
% -----
for l=1:Nbtri

    % Coordonnees des sommets du triangles
    % A COMPLETER
    S1= Coorneu(Numtri(l,1),:);
    S2= Coorneu(Numtri(l,2),:);
    S3= Coorneu(Numtri(l,3),:);

    % Calcul des matrices elementaires du triangle l
    Kel=matK_elem_p2(S1, S2, S3);
    Mel=matM_elem_p2(S1, S2, S3);

    % On fait l'assemblage des matrices globales
    for i=1:6
        I=Numtri(l,i);
        for j=1:6
            J=Numtri(l,j);
            MM(I,J) = MM(I,J)+ Mel(i,j);
            KK(I,J) = KK(I,J)+ Kel(i,j);
        end
    end
end % for l
```

Figure 8: Code pour l'assemblage des matrices élémentaires

2.6 Calcul du second membre

On peut trouver la solution $\mathbb{U}$ en résolvant le système linéaire :

$$(\mathbb{K} + \mathbb{M})\vec{U} = \vec{L}$$

On sait que :

$$\forall I \in \{1, 2, \dots, N\}, L_i = l(w_i) = \int_{\Omega} f w_i \, d\Omega$$

On peut utiliser la méthode d'interpolation : Si $f \in C^0(\bar{\Omega})$, on l'approche par son interpolée $\pi_h f = \sum_{j=1}^{Nbpt} f(M_j) w_j$ où $Nbpt$ est le nombre de sommets et de milieux d'arêtes.

Soit $i \in \{1, 2, \dots, N\}$, on a :

$$L_i \simeq \int_{\Omega} \pi_h f w_i \, d\Omega = \sum_{j=1}^{Nbpt} f(M_j) \int_{\Omega} w_j w_i \, d\Omega = \sum_{j=1}^{Nbpt} f(M_j) \mathbb{M}_{ji} = (\mathbb{M}\vec{F})_i \text{ où } \vec{F}_i = f(M_i)$$

d'où $\boxed{\vec{L} = \mathbb{M}\vec{F}}$

Dans le main (`principal_neumann_p2.m`) du code Matlab, on a alors :

```

% Calcul du second membre
% -----
% utiliser la routine f.m
FF = zeros(Nbpt,1);
for i=1:Nbpt
    FF(i) = f(Coorneu(i,1), Coorneu(i,2), 1);
end
LL = MM * FF;

% Résolution du système linéaire
% -----
UU = (MM+KK)\LL;

```

Figure 9: Code pour le calcul du second membre et résolution du système linéaire

On obtient ce résultat pour une simple vérification avec $f = 1$ et $h = 0.2$:

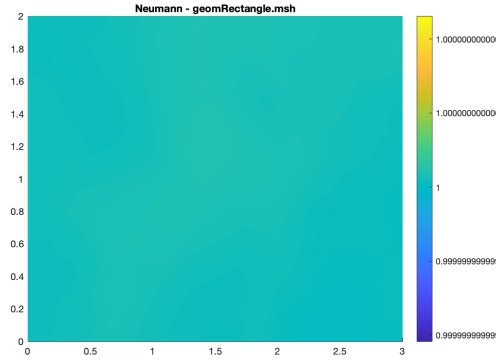


Figure 10: Solution approchée u_h pour $f = 1$ et $h = 0.2$

Il est évident que pour $f = 1$, la solution attendue de $(\mathbb{M} + \mathbb{K})\vec{U} = \vec{L}$ est $U_i = u_h(M_i) = 1 \forall i$. En affichant la solution numérique de $\vec{U}$ dans Matlab, on obtient exactement ce qui est attendu, une solution uniforme égale à 1 partout.

2.7 Validation de la méthode

On veut vérifier que le code calcule une solution approchée u_h correcte. Pour cela, on résout le problème (1) avec une solution connue u

$$\forall (x, y) \in \Omega, u(x, y) = 3 \cos(\pi x) \cos(2\pi y),$$

On a :

$$\Delta u = \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right) = -3\pi^2 \cos(\pi x) \cos(2\pi y) - 12\pi^2 \cos(\pi x) \cos(2\pi y) = -15\pi^2 \cos(\pi x) \cos(2\pi y)$$

Comme $u - \Delta u = f$, on a :

$$\boxed{\forall (x, y) \in \Omega, f(x, y) = (3 + 15\pi^2) \cos(\pi x) \cos(2\pi y)}$$

Voici le code Matlab de la fonction $f.m$, avec les différentes valeurs que l'on choisit en mettant en argument le choix de l'exercice que l'on traite :

```

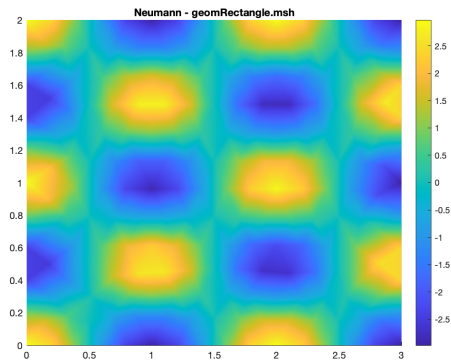
% exo vaut 0 pour l'essai en f = 1, 1 pour l'exo 1,
% 2 pour l'exo 2 homogène, 3 pour l'exo 2 non homogène
% sinon la valeur sera celle par défaut (ie 1)
function val = f(x,y, exo)
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Evaluation de la fonction second membre.
%
% SYNOPSIS val = f(x,y)
%
% INPUT * x,y : les 2 coordonnees du point ou on veut evaluer la fonction.
%
% OUTPUT - val: valeur de la fonction sur ce point.
%
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Exo 1 Vérification :
val = 1
if exo == 0
    val = 1;
% Exo 1 validation :
elseif exo == 1
    val = (3+15*pi*pi)*cos(pi*x)*cos(2*pi*y);
% Exo 2 validation homogène :
elseif exo == 2
    val = (3+6*pi*pi)*sin(pi*x)*sin(pi*y);
% Exo 2 validation non homogène :
elseif exo == 3
    val = (3+15*pi*pi)*cos(pi*x)*cos(2*pi*y);
end
end

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%24
%
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

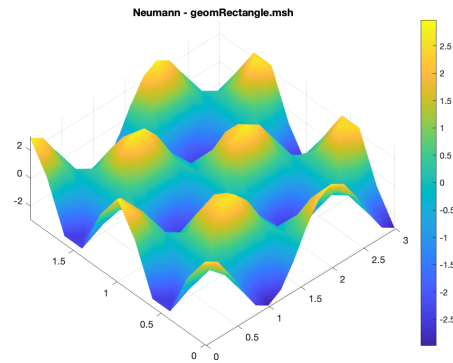
```

Figure 11: Code pour la fonction $f.m$

On trouve alors :

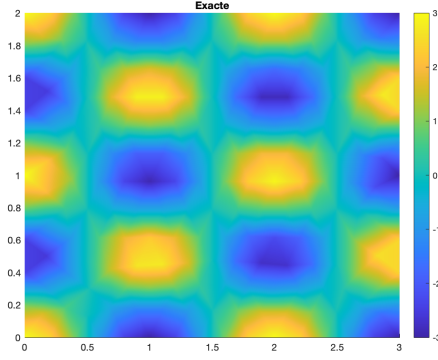


(a) Solution approchée

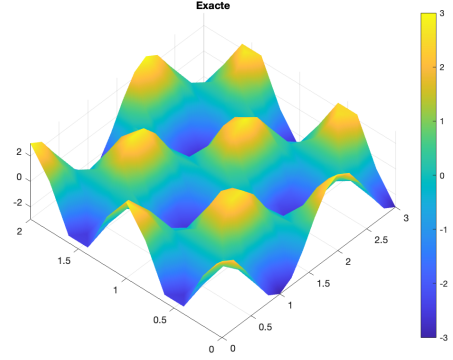


(b) Solution approchée en 3D

Figure 12: Résultats approchées pour la validation pour $h = 0.2$



(a) Solution exacte



(b) Solution exacte en 3D

Figure 13: Résultats exactes pour la validation pour $h = 0.2$

Les deux solutions semblent être les mêmes, pour les différencier et les comparer, on va effectuer une comparaison de la solution exacte et de la solution de neumann en utilisant la norme L^2 de la différence des deux. On va ici encore interpoler u par $\pi_h u = \sum_{j=1}^{Nbpt} u(M_j) w_j$ où $Nbpt$ est le nombre de sommets et de milieux d'arêtes.

$$\begin{aligned} \|\pi_h u - u_h\|_{L^2\Omega} &= \left(\int_{\Omega} (\pi_h u - u_h)^2 d\Omega \right)^{1/2} \\ &= \left(\int_{\Omega} \left(\sum_{i=1}^{Nbpt} (u - u_h)(M_i) w_i \right) \left(\sum_{j=1}^{Nbpt} (u - u_h)(M_j) w_j \right) d\Omega \right)^{1/2} \\ &= \left(\sum_{i=1}^{Nbpt} \sum_{j=1}^{Nbpt} (u - u_h)(M_i) \left(\int_{\Omega} w_i w_j d\Omega \right) (u - u_h)(M_j) \right)^{1/2} \end{aligned}$$

On a alors :

$$\|u - u_h\|_{L^2\Omega} \simeq \|\pi_h u - u_h\|_{L^2\Omega} = \left((\vec{U}_{exact} - \vec{U})^T \mathbb{M} (\vec{U}_{exact} - \vec{U}) \right)^{1/2}$$

De même :

$$\|u\|_{L^2\Omega} \simeq \|\pi_h u\|_{L^2\Omega} = \left((\vec{U}_{exact})^T \mathbb{M} (\vec{U}_{exact}) \right)^{1/2}$$

On peut alors calculer l'erreur relative :

$$\frac{\|u - u_h\|_{L^2\Omega}}{\|u\|_{L^2\Omega}} \simeq \frac{\left((\vec{U}_{exact} - \vec{U})^T \mathbb{M} (\vec{U}_{exact} - \vec{U}) \right)^{1/2}}{\left((\vec{U}_{exact})^T \mathbb{M} (\vec{U}_{exact}) \right)^{1/2}}$$

De manière analogue, on obtient pour la semi-norme H^1 :

$$|u - u_h|_{H^1\Omega} \simeq |\pi_h u - u_h|_{H^1\Omega} = \left((\vec{U}_{exact} - \vec{U})^T \mathbb{K} (\vec{U}_{exact} - \vec{U}) \right)^{1/2}$$

Et l'erreur relative :

$$\frac{|u - u_h|_{H^1\Omega}}{|u|_{H^1\Omega}} \simeq \frac{\left((\vec{U}_{exact} - \vec{U})^T \mathbb{K} (\vec{U}_{exact} - \vec{U}) \right)^{1/2}}{\left((\vec{U}_{exact})^T \mathbb{K} (\vec{U}_{exact}) \right)^{1/2}}$$

On implémente cela dans le code Matlab :

```
% validation seulement si voulu
% -----
% On initialise quand même les erreurs :
erreur_L2 = 0;
erreur_relative_L2 = 0;
erreur_semi_H1 = 0;
erreur_relative_semi_H1 = 0;
if strcmp(validation, 'oui')
    UU_exact = 3*cos(pi*Coorneu(:,1)).*cos(2*pi*Coorneu(:,2));
    if result_visualisation == 1
        affiche_ordre2(UU_exact, Numtri, Coorneu, "Exacte");
    end
    % Calcul de l'erreur L2
    erreur_L2 = sqrt((UU_exact-UU)'*MM*(UU_exact-UU));
    % Calcul de l'erreur relative L2
    erreur_relative_L2 = sqrt((UU_exact-UU)'*MM*(UU_exact-UU)) / sqrt((UU_exact)'*MM*(UU_exact));
    % Calcul de l'erreur semi H1
    erreur_semi_H1 = sqrt((UU_exact-UU)'*KK*(UU_exact-UU));
    % Calcul de l'erreur relative semi H1
    erreur_relative_semi_H1 = sqrt((UU_exact-UU)'*KK*(UU_exact-UU)) / sqrt((UU_exact)'*KK*(UU_exact));
end
```

Figure 14: Code pour le calcul des erreurs

On peut maintenant calculer ces erreurs pour différentes valeurs de h que l'on choisira de telle manière qu'environ $h_{i+1} = \frac{h_i}{2}$ afin de pouvoir effectuer une échelle $\log - \log$. Pour cela, j'ai dû modifier le fichier `principal_neumann_p2.m` pour qu'il prenne en argument la valeur de h , ainsi que si on veut avoir l'étape de validation, mais aussi si on veut afficher les différents graphiques et maillages trouvés.

```
% h_wanted = la taille de maillage voulue
% validation = 'oui' si on veut regarder les erreurs, 'non' sinon
% init_visualisation = 1 si on veut voir le maillage de départ, 0 sinon
% result_visualisation = 1 si on veut voir le résultat : en Neumann et en
% Exact
function [erreur_L2, erreur_relative_L2, erreur_semi_H1, erreur_relative_semi_H1] = principal_neumann_p2(h_wanted, validation, init_visualisation, result_visu)
%
% Changement de la valeur de h avant de calculer le tout
% -----
h_change_maillage(h_wanted);
```

Figure 15: Programme neumann modifié pour les erreurs

Un grand changement ici est l'intégration d'une fonction qui change directement la valeur de h dans le maillage sans avoir à le faire manuellement à chaque fois, en repassant par l'étape de génération du maillage, qui me nécessitait à chaque fois, d'ouvrir un fichier text pour modifier la valeur de h dans le fichier `.geo` et ensuite de créer le nouveau fichier `.msh` dans l'application dédiée. Ici tout est automatisée. Cependant, il faudra sûrement changer le chemin d'accès à l'outil `gmsh` selon comment et où il est installé sur votre machine. De plus, pour chaque fichier

```
function h_change_maillage(new_h_value)
% Lire le fichier .geo
filename = 'geomRectangle.geo';
fileContents = fileread(filename);

% Remplacer la valeur de h par la nouvelle valeur souhaitée
pattern = 'h = [0-9]*\.[0-9]*'; % Correspond à la ligne définissant h
replacement = sprintf('h = %.2f', new_h_value); % Remplace par la nouvelle valeur de h

% Modifier le contenu du fichier
newFileContents = regexp(fileContents, pattern, replacement);

% Écrire les modifications dans le fichier
fid = fopen(filename, 'w');
fwrite(fid, newFileContents);
fclose(fid);

% Exécuter Gmsh pour générer le maillage du geo au msh
% J'ai téléchargé par homebrew, le path est à changer selon la localisation
% de l'outil dans le système
system('/opt/homebrew/bin/gmsh geomRectangle.geo -2 -order 2 -o geomRectangle.msh');
end
```

Figure 16: Fonction pour changer la valeur de h dans le maillage

On crée ensuite un programme main qui va faire tourner le programme principal pour différentes valeurs de h et qui va stocker les erreurs dans un fichier texte pour pouvoir les afficher ensuite comme fait dans le TP1, en ajoutant des régressions linéaires.

```
% Les 4 tailles de maillage que je souhaite tester
h = [0.5, 0.25, 0.1, 0.05];
erreur_relative_L2 = [0, 0, 0, 0];
erreur_relative_semi_H1 = [0, 0, 0, 0];
for i=1:4
    [erreur_L2_tested, erreur_relative_L2_tested, erreur_semi_H1_tested, erreur_relative_semi_H1_tested] = principal_neumann_p2(h(i), 'oui', 0, 0);
    erreur_relative_L2(i) = erreur_relative_L2_tested;
    erreur_relative_semi_H1(i) = erreur_relative_semi_H1_tested;
end
```

Figure 17: Obtenir les erreurs relatives

```
% Abscisses en log(1/x)
log_x = log(1 ./ h);
% Ordonnées en log
log_y_L2 = log(erreur_relative_L2);
log_y_semi_H1 = log(erreur_relative_semi_H1);

% Créer la figure
figure;

% Graphique pour erreur_relative_L2
plot(log_x, log_y_L2, 'bo-', 'LineWidth', 1, 'MarkerSize', 8);
hold on;

% Graphique pour erreur_relative_semi_H1
plot(log_x, log_y_semi_H1, 'ro-', 'LineWidth', 1, 'MarkerSize', 8);

% Régressions linéaires
% L2
p_L2 = polyfit(log_x, log_y_L2, 1);
y_fit_L2 = polyval(p_L2, log_x);
plot(log_x, y_fit_L2, 'b--', 'LineWidth', 2);
% Semi H1
p_semi_H1 = polyfit(log_x, log_y_semi_H1, 1);
y_fit_semi_H1 = polyval(p_semi_H1, log_x);
plot(log_x, y_fit_semi_H1, 'r--', 'LineWidth', 2);

% Titre et étiquettes
xlabel('log(1/h)');
ylabel('log(erreur)');
title('Graph of log(1/h) vs. log(erreur)');
grid on;

% Ajouter une légende pour distinguer les courbes avec les coefficients
legend_str_L2 = sprintf('erreur\relative\L2 (Coef: %.4f)', p_L2(1));
legend_str_semi_H1 = sprintf('erreur\relative\semi_H1 (Coef: %.4f)', p_semi_H1(1));
legend(legend_str_L2, legend_str_semi_H1, 'Location', 'northeast');
hold off;
```

Figure 18: Régressions linéaires et graphiques associés

On obtient alors les résultats suivants pour différentes tailles de maillage h :

h	$\ u - u_h\ _{L^2(\Omega)}$	$\frac{\ u - u_h\ _{L^2(\Omega)}}{\ u\ _{L^2(\Omega)}}$	$ u - u_h _{H^1(\Omega)}$	$\frac{ u - u_h _{H^1(\Omega)}}{ u _{H^1(\Omega)}}$
0.5	0.4973	0.1470	2.7900	0.1125
0.2	0.0101	0.0027	0.1890	0.0073
0.1	0.0009	0.0002	0.0410	0.0016
0.05	0.0001	0.0000	0.0045	0.0002

Table 1: Tableau des résultats des erreurs pour l'étape de validation du problème de Neumann

On place alors les points sur un graphique et trace les régressions linéaires pour les erreurs relatives en fonction de h :

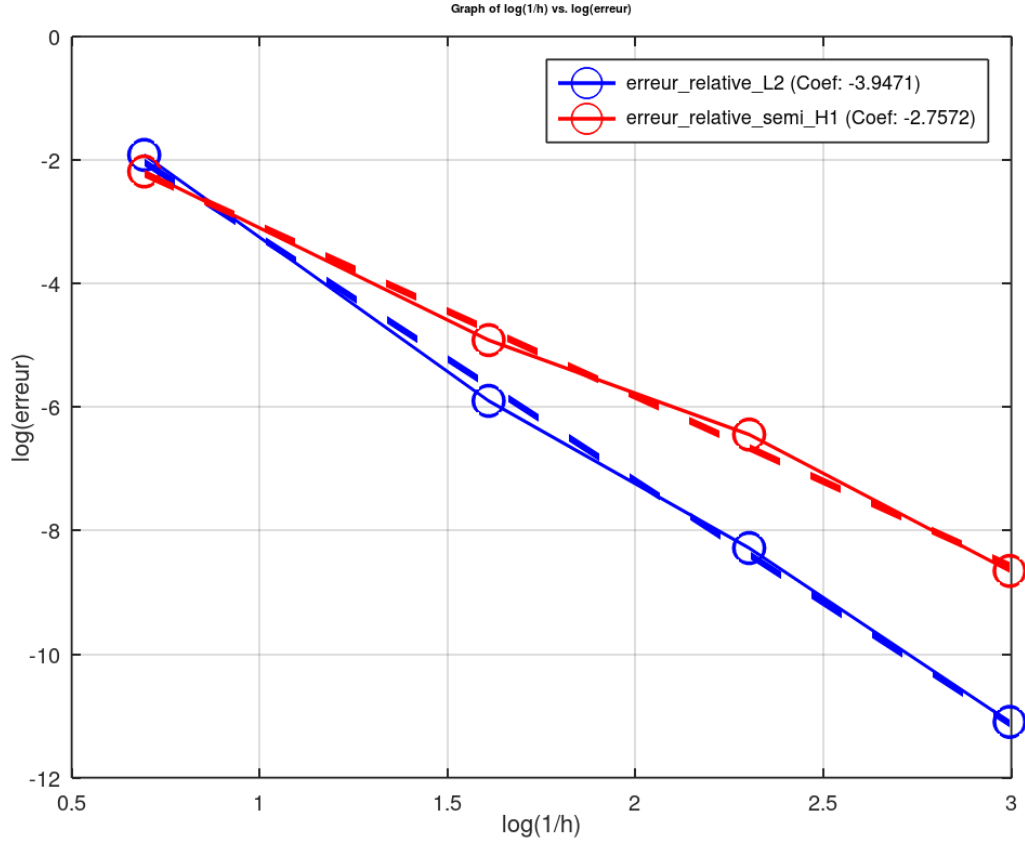


Figure 19: Graphique des erreurs pour la validation du problème de Neumann

C'est un résultat que l'on attend, un maillage plus fin donne des résultats plus précis, mais il nécessite également un plus grand temps de calculs informatiques, d'autant que la résolution d'ordre 2 est plus coûteuse en temps de calcul que la résolution d'ordre 1. Un autre résultat intéressant à noter, c'est que l'erreur converge plus rapidement à l'ordre 2 que pour l'ordre 1. En effet, les coefficients sont deux fois plus grands pour le $\log$ de l'erreur, montrant bien la convergence se faisant en h^2 à l'ordre 2, en h à l'ordre 1, ce que l'on attend.

Pour la méthode des éléments finis P^1 , la convergence de la solution numérique est donnée par l'estimation suivante :

$$\|u - u_h\|_E \leq Ch$$

où u est la solution exacte, u_h est la solution approchée, $\|\cdot\|_E$ est la norme appropriée (souvent la norme L^2), et C est une constante indépendante du maillage h .

Pour la méthode des éléments finis P^2 , la convergence est généralement de deuxième ordre :

$$\|u - u_h\|_E \leq Ch^2$$

où la notation est la même que précédemment, et l'erreur diminue plus rapidement avec la taille du maillage.

Par le passage au log, on observe bien l'apparition des coefficients 2, que l'on trouve bien dans les graphiques.

3 Éléments finis P^2 pour le problème de Poisson avec condition de Dirichlet

3.1 Conditions de Dirichlet homogènes

Nous considérons maintenant un problème de Poisson avec une condition au bord de Dirichlet homogène, le problème est le suivant :

Trouver $u \in H^1(\Omega)$ tel que

$$\begin{cases} u - \Delta u = f & \text{dans } \Omega \\ u = 0 & \text{sur } \partial\Omega \end{cases} \quad (13)$$

Nous avons vu dans le cours que la formulation variationnelle du problème s'écrit :

Trouver $u \in H_0^1(\Omega)$ tel que

$$\int_{\Omega} uv \, d\Omega + \int_{\Omega} \nabla u \cdot \nabla v \, d\Omega = \int_{\Omega} fv \, d\Omega, \quad \forall v \in H_0^1(\Omega), \quad (14)$$

et qu'elle est bien posée.

Nous reprenons les notations de la section précédente. Pour définir une approximation interne P^2 de $H_0^1(\Omega)$, on procède de la façon suivante : si on suppose que les nœuds (sommets et milieux) de la frontière $\partial\Omega$ sont numérotés de $N_0 + 1$ à N et que les nœuds à l'intérieur sont numérotés de 1 à N_0 , on définit :

$$V_h^0 = \text{Vect}(w_1, \dots, w_{N_0}).$$

Par construction, $V_h^0 \subset H_0^1(\Omega)$.

3.1.1 Formulation variationnelle discrète

Trouver $u_h \in V_h^0$ tel que $\forall v_h \in V_h^0, \int_{\Omega} (u_h v_h + \nabla u_h \cdot \nabla v_h) \, d\Omega = \int_{\Omega} f v_h \, d\Omega$

(15)

La solution approchée u_h s'écrit sous la forme :

$$\forall (x, y) \in \overline{\Omega}, \quad u_h(x, y) = \sum_{J=1}^{N_0} u_h(M_J) w_J(x, y)$$

On injecte cette expression dans l'équation variationnelle discrète et on obtient :

$$\sum_{I=1}^{N_0} u_h(M_I) \int_{\Omega} (w_I v_h + \nabla w_I \cdot \nabla v_h) \, d\Omega = \int_{\Omega} f v_h \, d\Omega$$

On choisit en particulier $v_h = w_J$ pour $J \in \{1, 2, \dots, N_0\}$, on a alors :

$$\sum_{I=1}^{N_0} u_h(M_I) \int_{\Omega} (w_I w_J + \nabla w_I \cdot \nabla w_J) \, d\Omega = \int_{\Omega} f w_J \, d\Omega$$

La nouvelle formulation variationnelle discrète est alors :

Trouver $\vec{U}^0 = \begin{pmatrix} u_h(M_1) \\ u_h(M_2) \\ \vdots \\ u_h(M_{N_0}) \end{pmatrix}$ tel que $(\mathbb{M}^0 + \mathbb{K}^0) \vec{U} = \vec{L}^0$

où $\forall (I, J) \in \{1, 2, \dots, N_0\}^2, \mathbb{M}_{IJ}^0 = \int_{\Omega} w_I w_J \, d\Omega, \mathbb{K}_{IJ}^0 = \int_{\Omega} \nabla w_I \cdot \nabla w_J \, d\Omega, \text{ et } \vec{L}^0_I = \int_{\Omega} f w_I \, d\Omega$

(16)

$\mathbb{M}^0$ est symétrique :

$$\mathbb{M}_{IJ}^0 = \int_{\Omega} w_I w_J \, d\Omega = \int_{\Omega} w_J w_I \, d\Omega = \mathbb{M}_{JI}^0$$

$\mathbb{M}^0$ est positive :

Soit $\vec{U}^0 \in \mathbb{R}^{N_0}$, $I \in \{1, 2, \dots, N_0\}$, on a :

$$(\mathbb{M}^0 \vec{U}^0)_I = \sum_{J=1}^{N_0} \mathbb{M}_{IJ}^0 u_h(M_J) = \sum_{J=1}^{N_0} \left(\int_{\Omega} w_I w_J d\Omega \right) u_h(M_J) = \int_{\Omega} w_I \left(\sum_{J=1}^{N_0} w_J u_h(M_J) \right) d\Omega$$

Donc :

$$\begin{aligned} \vec{U}^0{}^T \mathbb{M}^0 \vec{U}^0 &= \sum_{I=1}^N u_h(M_I) (\mathbb{M}^0 \vec{U}^0)_I \\ &= \sum_{I=1}^N u_h(M_I) \left(\int_{\Omega} w_I \left(\sum_{J=1}^{N_0} w_J u_h(M_J) \right) d\Omega \right) \\ &= \int_{\Omega} \left(\sum_{I=1}^N w_I u_h(M_I) \right)^2 d\Omega \\ &\geq 0 \end{aligned}$$

$\mathbb{K}^0$ est symétrique positive de manière analogue à $\mathbb{M}^0$:

Conclusion : $\boxed{\mathbb{M}^0, \mathbb{K}^0 \in S_n^+(\mathbb{R})}$

3.1.2 Résolution informatique

Dans la pratique, plutôt que de résoudre le système linéaire, on préfère résoudre l'équation matricielle

$$\tilde{\mathbb{A}} \vec{U} = \tilde{L} \quad (17)$$

obtenue à l'aide de la technique de *pseudo-élimination*. On a toujours $\tilde{\mathbb{A}} \in \mathbb{R}^{N \times N}$ et $\tilde{L} \in \mathbb{R}^N$.

En supposant que les sommets et les milieux d'arêtes au bord apparaissent en premier dans la numérotation des nœuds, on a :

$$\tilde{\mathbb{A}} = \begin{pmatrix} \mathbb{A}_{BB} & \mathbb{A}_{BI} \\ \mathbb{A}_{IB} & \mathbb{A}_{II} \end{pmatrix}, \vec{U} = \begin{pmatrix} \vec{U}_B \\ \vec{U}_I \end{pmatrix} \text{ et } \tilde{L} = \begin{pmatrix} \vec{L}_B \\ \vec{L}_I \end{pmatrix}$$

où les notations sont les suivantes :

- $\mathbb{A}_{BB} \in \mathbb{M}_{N-N_0}(\mathbb{R}) = (a(w_i, w_j))_{M_i \in \partial\Omega, M_j \in \partial\Omega}$
- $\mathbb{A}_{BI} \in \mathbb{M}_{(N-N_0) \times N_0}(\mathbb{R}) = (a(w_i, w_j))_{M_i \in \partial\Omega, M_j \notin \partial\Omega}$
- $\mathbb{A}_{IB} \in \mathbb{M}_{N_0 \times (N-N_0)}(\mathbb{R}) = (a(w_i, w_j))_{M_i \notin \partial\Omega, M_j \in \partial\Omega}$
- $\mathbb{A}_{II} \in \mathbb{M}_{N_0}(\mathbb{R}) = (a(w_i, w_j))_{M_i \notin \partial\Omega, M_j \notin \partial\Omega}$

On se retrouve alors avec le système suivant, en simplifiant les matrices et vecteurs :

$$\boxed{\begin{aligned} \text{Trouver } \vec{U} &= \begin{pmatrix} u_h(M_1) \\ u_h(M_2) \\ \vdots \\ u_h(M_N) \end{pmatrix} \text{ tel que } \tilde{\mathbb{A}} \vec{U} = \tilde{L} \\ \text{où } \tilde{\mathbb{A}} &= \begin{pmatrix} I_{(N-N_0)} & 0_{N_0} \\ 0_{N-N_0} & \mathbb{A}^0 \end{pmatrix}, \tilde{L} = \begin{pmatrix} \vec{0} \\ \vec{L}^0 \end{pmatrix} \end{aligned}} \quad (18)$$

En pratique, on va utiliser la méthode de pseudo-élimination pour résoudre ce système linéaire, les sommets et milieux d'arêtes au bord n'apparaissant pas en premier dans la numérotation des nœuds. Voici le code Matlab d'une première version de la fonction `elimine.m` qui permet d'effectuer sur la matrice $\tilde{\mathbb{A}}$ et le vecteur $\tilde{L}$ la pseudo-élimination :

```

function [AA_tilde, LL_tilde] = elimine(AA, LL, Refneu)

    AA_tilde = AA;
    LL_tilde = LL;
    Nbpt = size(Refneu,1);
    for i=1:Nbpt
        % pour un sommet sur le bord
        if Refneu(i) == 1
            AA_tilde(i,:) = 0;
            AA_tilde(:,i) = 0;
            AA_tilde(i,i) = 1;
            LL_tilde(i) = 0;
        end
    end
end

```

Figure 20: Code pour la pseudo-élimination Dirichlet homogène

Voici le code utilisé pour cela dans un fichier nommé `principal_dirichlet_p2.m` qui marche pour le problème homogène et le problème non homogène :

```

% =====
%
% une routine pour la mise en oeuvre des EF P2 Lagrange
% pour l'équation de Laplace suivante, avec conditions de Dirichlet
%
% | -\Delta u + u = f,   dans \Omega
% |      u = 0,   sur le bord de \Omega si homogène
% |      u = g,   sur le bord de \Omega si non homogène
%
% Le script est vu comme une fonction pour calculer les solutions, qui sera
% appelé dans un main pour éviter d'appeler à la main pour faire les
% régressions linéaires sur les erreurs
% =====
% h_wanted = la taille de maillage voulue
% validation = 'oui' si on veut regarder les erreurs, 'non' sinon
% init_visualisation = 1 si on veut voir le maillage de départ, 0 sinon
% result_visualisation = 1 si on veut voir le résultat : en approchée et en Exact
% is_homogene : 1 si on résout le problème homogène, 0 pour le problème non homogène
function [erreur_L2, erreur_relative_L2, erreur_semi_H1, erreur_relative_semi_H1] =
principal_dirichlet_p2(h_wanted, validation, init_visualisation, result_visualisation, is_homogene)

```

Figure 21: Code principal pour Dirichlet

La suite du code est la même que pour le problème de Neumann jusqu'à la résolution du système linéaire, qui nécessite d'appliquer la méthode de pseudo-élimination.

```

% Faire la pseudo élimination
% -----
[AA_tilde, LL_tilde] = elimine(MM+KK, LL, Refneu, Coorneu, is_homogene);

% Résolution du système linéaire
% -----
UU = AA_tilde\LL_tilde;

```

Figure 22: Code pour la pseudo-élimination dans le programme principal

3.1.3 Validation

On veut vérifier que le code calcule une solution approchée u_h correcte. Pour cela, on résout le problème avec une solution u connue égale à $u(x, y) = 3 \sin(\pi x) \sin(\pi y)$, pour $(x, y) \in \overline{\Omega} = [0, 3] \times [0, 2]$. Vérifions d'abord que u vérifie bien la condition au bord du problème :

$$\begin{aligned}
 \forall x \in [0, 3], \quad u(x, 0) &= 3 \sin(\pi x) \sin(0) = 0 \\
 \forall x \in [0, 3], \quad u(x, 2) &= 3 \sin(\pi x) \sin(2\pi) = 0 \\
 \forall y \in [0, 2], \quad u(0, y) &= 3 \sin(0) \sin(\pi y) = 0 \\
 \forall y \in [0, 2], \quad u(3, y) &= 3 \sin(3\pi) \sin(\pi y) = 0
 \end{aligned}$$

La condition au bord est bien vérifiée.
Calculons maintenant f :

$$\begin{aligned}\Delta u &= \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \\ &= -3\pi^2 \sin(\pi x) \sin(\pi y) - 3\pi^2 \sin(\pi x) \sin(\pi y) \\ &= -6\pi^2 \sin(\pi x) \sin(\pi y)\end{aligned}$$

Donc comme $u - \Delta u = f$, on a :

$$\boxed{\forall (x, y) \in \Omega, f(x, y) = 3 \sin(\pi x) \sin(\pi y) + 6\pi^2 \sin(\pi x) \sin(\pi y) = (3 + 6\pi^2) \sin(\pi x) \sin(\pi y)}$$

Comme premiers résultats, on a pour $h = 0.2$:

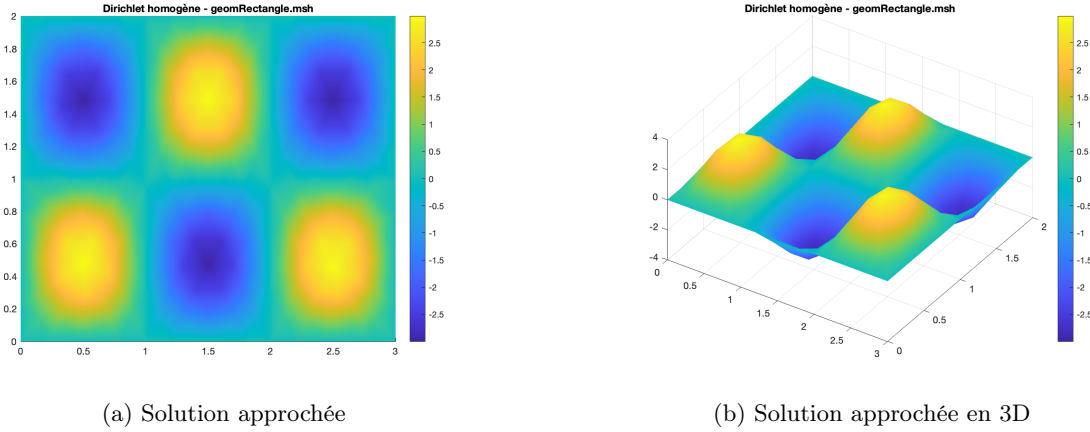


Figure 23: Résultats approchés pour la validation pour $h = 0.2$

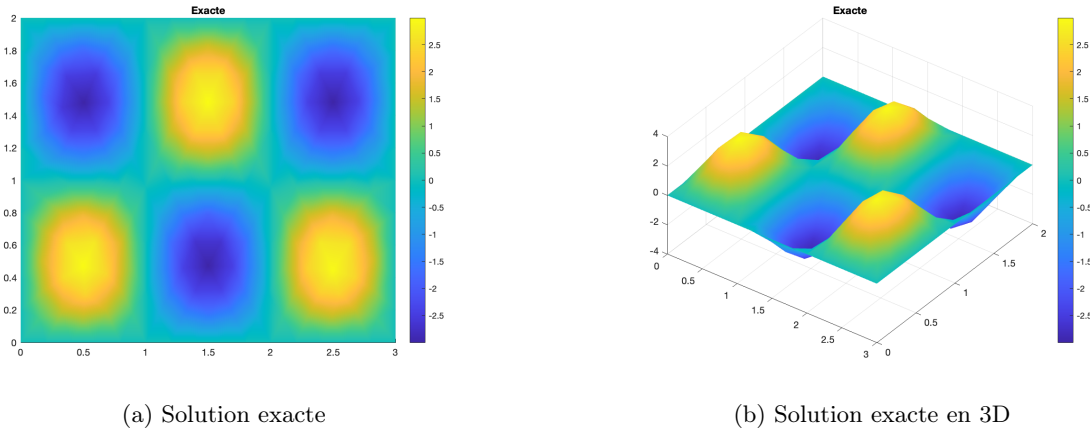


Figure 24: Résultats exactes pour la validation pour $h = 0.2$

Les deux solutions semblent être les mêmes, pour les différencier et les comparer, on va effectuer une comparaison de la solution exacte et de la solution de Neumann en des normes comme dans la partie 1 et dans le TP1.

On crée un autre fichier `validation_dirichlet_homogene.m` pour, comme en partie 1 et dans le TP1, calculer les erreurs relatives et les tracer en fonction de h , en effectuant des régressions linéaires pour les erreurs relatives en fonction de h (et de son log). On a juste à modifier une ligne par rapport à la version pour Neumann.

```

% Les 4 tailles de maillage que je souhaite tester
h = [0.5, 0.2, 0.1, 0.05];
erreur_L2 = [0, 0, 0, 0];
erreur_semi_H1 = [0, 0, 0, 0];
erreur_relative_L2 = [0, 0, 0, 0];
erreur_relative_semi_H1 = [0, 0, 0, 0];
for i=1:4
    [erreur_L2_tested, erreur_relative_L2_tested, erreur_semi_H1_tested, erreur_relative_semi_H1_tested] = principal_dirichlet_p2(h(i), 'oui', 0, 0, 1);
    erreur_L2(i) = erreur_L2_tested;
    erreur_semi_H1(i) = erreur_semi_H1_tested;
    erreur_relative_L2(i) = erreur_relative_L2_tested;
    erreur_relative_semi_H1(i) = erreur_relative_semi_H1_tested;
end

```

Figure 25: Code pour la validation du problème de Dirichlet homogène

On obtient alors les résultats suivants :

h	$\ u - u_h\ _{L^2(\Omega)}$	$\frac{\ u - u_h\ _{L^2(\Omega)}}{\ u\ _{L^2(\Omega)}}$	$ u - u_h _{H^1(\Omega)}$	$\frac{ u - u_h _{H^1(\Omega)}}{ u _{H^1(\Omega)}}$
0.5	0.0565	0.0156	0.4296	0.0265
0.2	0.0019	0.0005	0.0368	0.0023
0.1	0.0002	0.0000	0.0097	0.0006
0.05	0.0000	0.0000	0.0012	0.0001

Table 2: Tableau des résultats des erreurs

On place alors les points sur un graphique et trace les régressions linéaires pour les erreurs relatives en fonction de h :

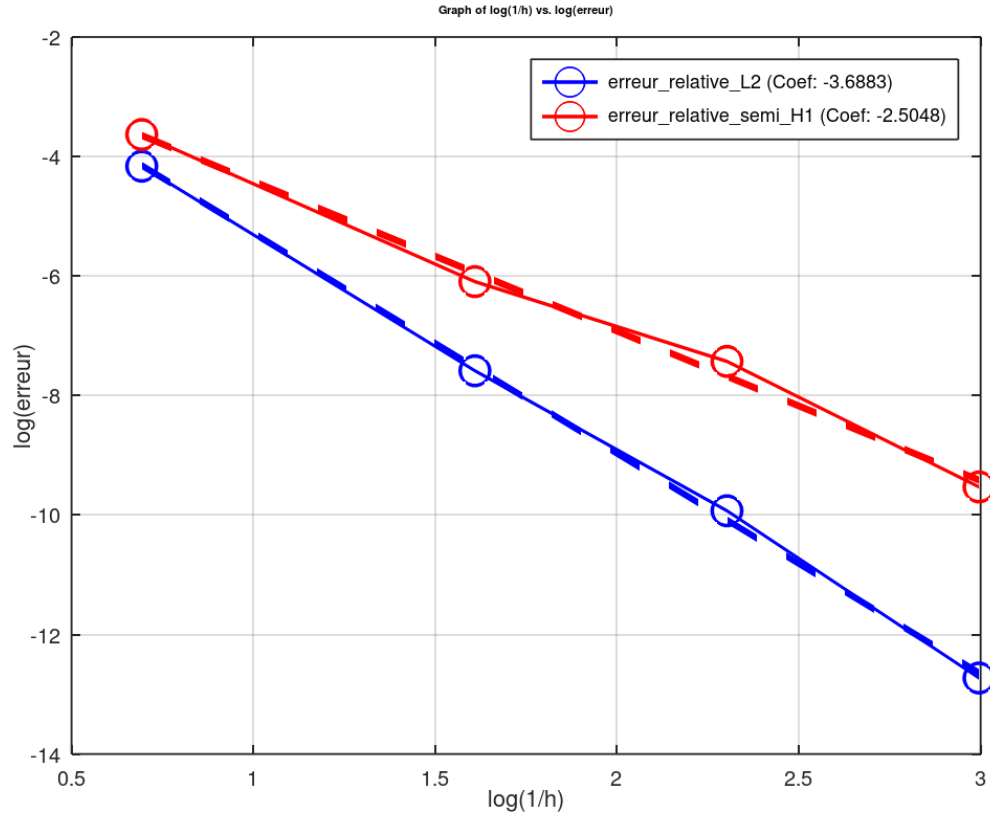


Figure 26: Graphique des erreurs pour la validation du problème de Dirichlet homogène

C'est un résultat que l'on attend, un maillage plus raffiné donne des résultats plus précis, mais il

nécessite également un plus grand temps de calculs informatiques. Les coefficients semblent aussi plutôt être cohérent et correspondent à ce que l'on recherche.

3.2 Conditions de Dirichlet non homogènes

3.2.1 Prise en compte de la non homogénéité du problème dans la pseudo-élimination

$$\begin{cases} u - \Delta u = f & \text{dans } \Omega \\ u = g & \text{sur } \partial\Omega \end{cases} \quad (19)$$

Nous avons vu dans le cours que la formulation variationnelle du problème peut s'écrire comme :
Trouver $u \in H^1(\Omega)$ avec $u|_{\partial\Omega} = g$ tel que

$$\int_{\Omega} uv \, d\Omega + \int_{\Omega} \nabla u \cdot \nabla v \, d\Omega = \int_{\Omega} fv \, d\Omega, \quad \forall v \in H_0^1(\Omega), \quad (20)$$

et qu'il est possible d'étendre la technique de pseudo-élimination à des conditions de Dirichlet non homogènes. Pour ce faire, en supposant que les noeuds de la frontière sont numérotés en premier, nous admettons qu'il suffit de modifier la matrice et le second membre issus de la formulation variationnelle assemblés avec toutes les fonctions de base

$$\begin{bmatrix} \mathbb{A}_{BB} & \mathbb{A}_{BI} \\ \mathbb{A}_{IB} & \mathbb{A}_{II} \end{bmatrix} \cdot \begin{bmatrix} \vec{U}_B \\ \vec{U}_I \end{bmatrix} = \begin{bmatrix} \vec{L}_B \\ \vec{L}_I \end{bmatrix}$$

En imposant $\mathbb{A}_{BI} = 0_{N_0}$, $\mathbb{A}_{BB} = I_{N-N_0}$, et $\vec{L}_B = \vec{g}$, où $\vec{g}$ est le vecteur colonne composé des valeurs prises par g aux noeuds M_J du bord, car il nous faut résoudre pour le problème non homogène :

$$\begin{cases} \mathbb{A}_{IB}\vec{U}_B + \mathbb{A}_{II}\vec{U}_I = \vec{L}_I \\ \vec{U}_B = \vec{g} \end{cases} \quad (21)$$

On remarquera qu'on n'a pas imposé $\mathbb{A}_{IB} = 0_{N-N_0}$, ce qui revient à ne pas annuler les colonnes dans `elimine.m`. Il faut aussi ajouter le fait que $\vec{L}_B$ est modifié pour prendre en compte les valeurs de $\vec{g}$ aux noeuds du bord. Le programme sera utilisé à la fois pour Dirichlet homogène et non homogène maintenant, il suffit alors de passer en argument le type de problème à résoudre :

```
function [AA_tilde, LL_tilde] = elimine(AA, LL, Refneu, Coorneu, is_homogene)

    AA_tilde = AA;
    LL_tilde = LL;
    Nbpt = size(Refneu,1);
    GG = zeros(Nbpt, 1);
    % LB vaut g pour Dirichlet non homogène, on charge alors g dans ce cas
    if is_homogene == 0
        GG = g(Refneu, Coorneu, Nbpt);
    end
    for i=1:Nbpt
        % pour un sommet sur le bord
        if Refneu(i) == 1
            AA_tilde(i,:) = 0;
            % on supprime la colonne pour Dirichlet homogène
            if is_homogene == 1
                AA_tilde(:,i) = 0;
            end
            AA_tilde(i,i) = 1;
            % GG vaut 0 si homogène et g sinon
            LL_tilde(i) = GG(i);
        end
    end
end
```

Figure 27: Code pour la pseudo-élimination et l'ajout de $\vec{g}$ en Dirichlet homogène et non homogène

3.2.2 Validation

On cherche à valider le code avec une solution u connue égale à $u(x, y) = 3 \cos(\pi x) \cos(2\pi y)$, pour $(x, y) \in \bar{\Omega} = [0, 3] \times [0, 2]$. Calculons f :

$$\begin{aligned}\Delta u &= \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \\ &= -3\pi^2 \cos(\pi x) \cos(2\pi y) - 12\pi^2 \cos(\pi x) \cos(2\pi y) \\ &= -15\pi^2 \cos(\pi x) \cos(2\pi y)\end{aligned}$$

Donc comme $u - \Delta u = f$, on a :

$$\forall (x, y) \in \Omega, f(x, y) = (3 + 15\pi^2) \cos(\pi x) \cos(2\pi y)$$

Maintenant, on cherche à calculer g pour que $u = g$ sur le bord. On a :

$$\begin{aligned}\forall x \in [0, 3], \quad g(x, 0) &= 3 \cos(\pi x) \cos(0) = 3 \cos(\pi x) \\ \forall x \in [0, 3], \quad g(x, 2) &= 3 \cos(\pi x) \cos(2\pi) = 3 \cos(\pi x) \\ \forall y \in [0, 2], \quad g(0, y) &= 3 \cos(0) \cos(2\pi y) = 3 \cos(2\pi y) \\ \forall y \in [0, 2], \quad g(3, y) &= 3 \cos(3\pi) \cos(2\pi y) = 3 \cos(2\pi y)\end{aligned}$$

On peut donc écrire la définir de g comme suit :

$$g : \begin{array}{ccc} \mathbb{R} \times \mathbb{R} & \longrightarrow & \mathbb{R} \\ (x, y) & \longmapsto & \begin{cases} 3 \cos(\pi x) & \text{si } y = 0 \text{ ou } y = 2 \\ 3 \cos(2\pi y) & \text{si } x = 0 \text{ ou } x = 3 \\ 0 & \text{sinon} \end{cases} \end{array}$$

On implémente le vecteur comme ceci, il sera utile dans la routine `elimine.m` pour ajouter cette composante au vecteur $\vec{L}$:

```
function [GG] = g(Refneu, Coorneu, Nbpt)
    GG = zeros(Nbpt, 1);
    for i=1:Nbpt
        % si on est sur le bord
        if Refneu(i) == 1
            if Coorneu(i,1) == 0 || Coorneu(i,1) == 3
                GG(i) = 3*cos(2*pi*Coorneu(i,2));
            end
            if Coorneu(i,2) == 0 || Coorneu(i,2) == 2
                GG(i) = 3*cos(pi*Coorneu(i,1));
            end
        end
    end
end
```

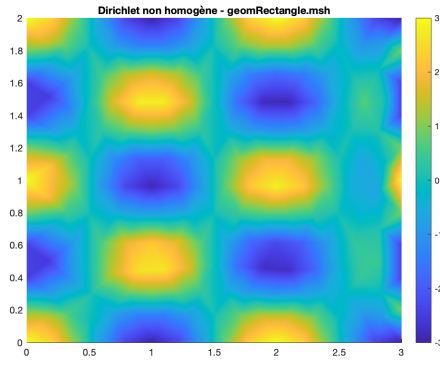
Figure 28: Vecteur $\vec{g}$ pour Dirichlet non homogène

On modifie aussi la fonction `exact` pour qu'elle dépende du choix de l'homogénéité de la condition de Dirichlet :

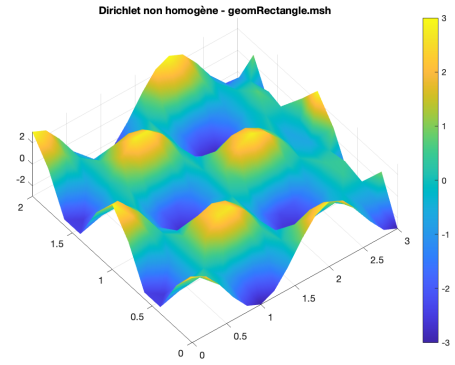
```
if strcmp(validation, 'oui')
    if is_homogene == 1
        UU_exact = 3*sin(pi*Coorneu(:,1)).*sin(pi*Coorneu(:,2));
    elseif is_homogene == 0
        UU_exact = 3*cos(pi*Coorneu(:,1)).*cos(2*pi*Coorneu(:,2));
    end
end
```

Figure 29: Changement du code de la fonction `exact` pour Dirichlet homogène et non homogène

Comme premiers résultats, on a pour $h = 0.2$:

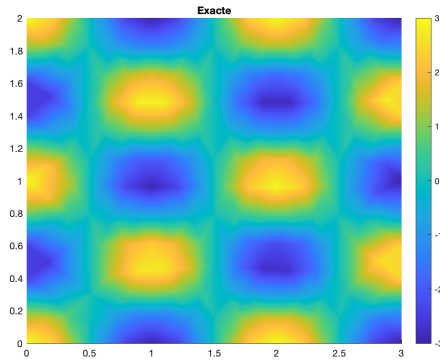


(a) Solution approchée

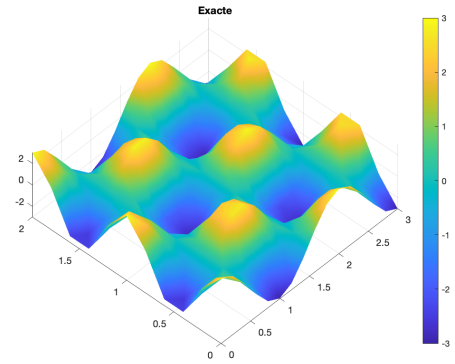


(b) Solution approchée en 3D

Figure 30: Résultats approchés pour la validation pour $h = 0.2$



(a) Solution exacte



(b) Solution exacte en 3D

Figure 31: Résultats exactes pour la validation pour $h = 0.2$

Les deux solutions semblent être presque les mêmes, bien qu'on puisse voir quelques différences à l'œil nu, pour les différencier et les comparer, on va effectuer une comparaison de la solution exacte et de la solution de Neumann en des normes comme dans la partie 1 et dans le TP1.

On crée un autre fichier `validation_dirichlet_non_homogene.m` qui fait comme `validation_dirichlet_homogene.m` mais pour le problème non homogène. On a juste à modifier une ligne.

```
% Les 4 tailles de maillage que je souhaite tester
h = [0.5, 0.2, 0.1, 0.05];
erreur_L2 = [0, 0, 0, 0];
erreur_semi_H1 = [0, 0, 0, 0];
erreur_relative_L2 = [0, 0, 0, 0];
erreur_relative_semi_H1 = [0, 0, 0, 0];
for i=1:4
    [erreur_L2_tested, erreur_relative_L2_tested, erreur_semi_H1_tested, erreur_relative_semi_H1_tested] = principal_dirichlet_p2(h(i), 'oui', 0, 0, 0);
    erreur_L2(i) = erreur_L2_tested;
    erreur_semi_H1(i) = erreur_semi_H1_tested;
    erreur_relative_L2(i) = erreur_relative_L2_tested;
    erreur_relative_semi_H1(i) = erreur_relative_semi_H1_tested;
end
```

Figure 32: Code pour la validation du problème de Dirichlet non homogène

On obtient alors les résultats suivants :

h	$\ u - u_h\ _{L^2(\Omega)}$	$\frac{\ u - u_h\ _{L^2(\Omega)}}{\ u\ _{L^2(\Omega)}}$	$ u - u_h _{H^1(\Omega)}$	$\frac{ u - u_h _{H^1(\Omega)}}{ u _{H^1(\Omega)}}$
0.5	1.4132	0.4178	11.984	0.4831
0.2	1.5913	0.4340	15.861	0.6150
0.1	1.5905	0.4329	16.891	0.6546
0.05	1.5910	0.4330	17.821	0.6904

Table 3: Tableau des résultats des erreurs

On place alors les points sur un graphique et trace les régressions linéaires pour les erreurs relatives en fonction de h :

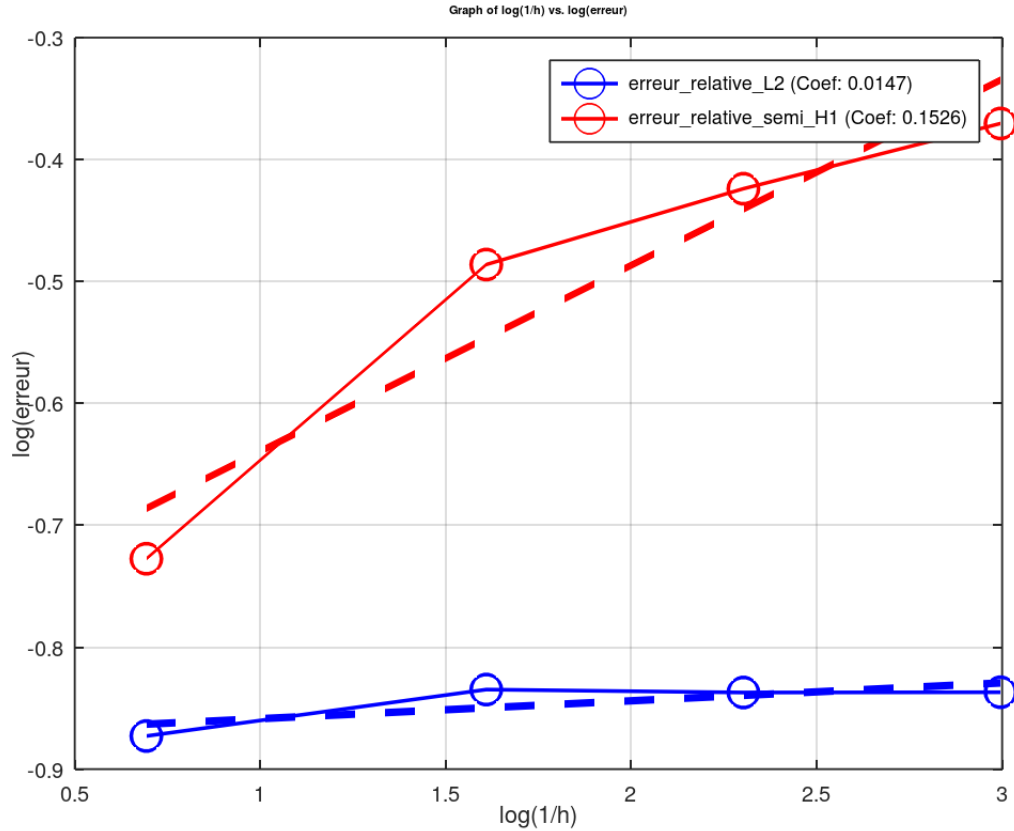


Figure 33: Graphique des erreurs pour la validation du problème de Dirichlet non homogène

Ici les ordre de grandeur du $\log$ de l'erreur sont très petites, quasiment insignifiante. Dans le cadre des équations de Dirichlet, les conditions non homogènes réduisent l'erreur numérique par rapport aux conditions homogènes en raison de la nature des contraintes sur le bord. Lorsque la condition sur la frontière est non homogène, c'est-à-dire $u = g$ sur $\partial\Omega$ avec g donné, la solution numérique est directement ajustée à cette fonction g sur le bord, ce qui permet une meilleure approximation de la solution exacte. En revanche, pour les conditions homogènes $u = 0$ sur $\partial\Omega$, la solution doit satisfaire à la fois aux équations différentielles et aux contraintes de Dirichlet sur le bord, ce qui peut introduire davantage d'erreur, notamment si la solution exacte présente des comportements complexes ou des singularités proches de la frontière. Ainsi, les conditions non homogènes favorisent une meilleure approximation dans les cas où la solution exacte est bien comportée sur le bord.

4 Équations de Stokes

Nous allons maintenant aborder le problème de Stokes. Soit Ω un ouvert borné de $\mathbb{R}^2$ à frontière polygonale et $\Gamma = \partial\Omega$. La frontière Γ est partitionnée en $\Gamma = \Gamma_D \cup \Gamma_N$, avec $\Gamma_N \neq \emptyset$. On impose une condition de Dirichlet sur Γ_D et une condition de Neumann sur Γ_N . On s'intéresse au calcul de la vitesse $\vec{u} = \vec{u}(x, y) = (u_1(x, y), u_2(x, y))$ et de la pression $p = p(x, y)$, solutions des équations de Stokes suivantes :

trouver $\vec{u} = (u_1, u_2) \in [H^1(\Omega)]^2$ et $p \in H^1(\Omega)$ tels que

$$\begin{cases} -\nu\Delta\vec{u} + \nabla p = 0 & \text{dans } \Omega \\ -\operatorname{div}(\vec{u}) = 0 & \text{dans } \Omega \\ \vec{u} = \vec{g} & \text{sur } \Gamma_D \\ \nu\frac{\partial\vec{u}}{\partial\vec{n}} - p\vec{n} = 0 & \text{sur } \Gamma_N \end{cases} \quad (22)$$

où la viscosité dynamique ν du fluide est constante et $\vec{g}$ est la donnée sur le bord de Dirichlet. Comme d'habitude, $\vec{n}$ désigne la normale unitaire extérieure à $\partial\Omega$. Le problème peut s'écrire composante par composante de la façon suivante :

$$\begin{cases} -\nu\Delta u_1 + \frac{\partial p}{\partial x} = 0 & \text{dans } \Omega \\ -\nu\Delta u_2 + \frac{\partial p}{\partial y} = 0 & \text{dans } \Omega \\ -\frac{\partial u_1}{\partial x} - \frac{\partial u_2}{\partial y} = 0 & \text{dans } \Omega \\ (u_1, u_2) = (g_1, g_2) & \text{sur } \Gamma_D \\ \nu\nabla u_1 \cdot \vec{n} - pn_1 = 0 & \text{sur } \Gamma_N \\ \nu\nabla u_2 \cdot \vec{n} - pn_2 = 0 & \text{sur } \Gamma_N \end{cases} \quad (23)$$

4.1 Obtention de la formulation variationnelle

Multiplions la première équation par $v_1 \in H^1(\Omega)$ et la deuxième par $v_2 \in H^1(\Omega)$, et intégrons sur Ω . Commençons par la première équation :

$$\int_{\Omega} \left(-\nu\Delta u_1 + \frac{\partial p}{\partial x} \right) v_1 \, d\Omega = 0$$

En intégrant par parties les termes de Laplacien (*et comme* $-\Delta u_1 = \frac{\partial p}{\partial x} \in L^2(\Omega)$), nous utilisons les formules de Green :

$$\int_{\Omega} \Delta u_1 v_1 \, d\Omega + \int_{\Omega} \nabla u_1 \nabla v_1 \, d\Omega = \int_{\partial\Omega} (\nabla u_1 \cdot \vec{n}) v_1 \, \partial\Omega$$

Et par parties :

$$\int_{\Omega} \frac{\partial p}{\partial x} v_1 \, d\Omega + \int_{\Omega} p \frac{\partial v_1}{\partial x} \, d\Omega = \int_{\partial\Omega} p v_1 n_1 \, \partial\Omega$$

On obtient alors :

$$\int_{\Omega} \nu \nabla u_1 \nabla v_1 \, d\Omega - \int_{\Omega} p \frac{\partial v_1}{\partial x} \, d\Omega = \int_{\partial\Omega} \nu (\nabla u_1 \cdot \vec{n}) v_1 \, \partial\Omega - \int_{\partial\Omega} p v_1 n_1 \, \partial\Omega$$

Que l'on peut écrire :

$$\nu \int_{\Omega} \nabla u_1 \nabla v_1 \, d\Omega - \int_{\Omega} p \frac{\partial v_1}{\partial x} \, d\Omega = \int_{\partial\Omega} (\nu (\nabla u_1 \cdot \vec{n}) - p n_1) v_1 \, \partial\Omega$$

De manière similaire, on obtient :

$$\nu \int_{\Omega} \nabla u_2 \nabla v_2 \, d\Omega - \int_{\Omega} p \frac{\partial v_2}{\partial x} \, d\Omega = \int_{\partial\Omega} (\nu(\nabla u_2 \cdot \vec{n}) - pn_2) v_2 \, \partial\Omega$$

En sommant les deux équations et en notant $\nabla \vec{u} : \nabla \vec{v} = \nabla u_1 \cdot \nabla v_1 + \nabla u_2 \cdot \nabla v_2$, on a :

$$\nu \int_{\Omega} \nabla \vec{u} : \nabla \vec{v} \, d\Omega - \int_{\Omega} p \operatorname{div}(\vec{v}) \, d\Omega = \int_{\partial\Omega} ((\nu(\nabla u_2 \cdot \vec{n}) - pn_2) v_2 + (\nu(\nabla u_1 \cdot \vec{n}) - pn_1) v_1) \, \partial\Omega$$

Pour calculer l'intégrale à droite, on utilise les conditions du problème en décomposant l'intégrale sur $\partial\Omega$ par la somme des intégrales sur Γ_D et Γ_N . En choisissant de plus $\vec{v}$ dans :

$$[H_{0,D}^1(\Omega)]^2 = \{\vec{v} \in [H^1(\Omega)]^2 \text{ tel que } \vec{v}|_{\Gamma_D} = (0, 0)\}.$$

Or :

$$\begin{cases} \nu \nabla u_1 \cdot \vec{n} - pn_1 = 0 & \text{sur } \Gamma_N \\ \nu \nabla u_2 \cdot \vec{n} - pn_2 = 0 & \text{sur } \Gamma_N \\ v_1 = 0 & \text{sur } \Gamma_D \\ v_2 = 0 & \text{sur } \Gamma_D \end{cases}$$

Donc :

$$\int_{\partial\Omega} ((\nu(\nabla u_2 \cdot \vec{n}) - pn_2) v_2 + (\nu(\nabla u_1 \cdot \vec{n}) - pn_1) v_1) \, \partial\Omega = 0$$

Pour compléter le résultat, soit $q \in L^2(\Omega)$ et comme $-\frac{\partial u_1}{\partial x} - \frac{\partial u_2}{\partial y} = 0$ dans Ω , on multiplie cette équation par q et on intègre selon Ω :

$$-\int_{\Omega} \operatorname{div}(\vec{u}) q \, d\Omega = 0$$

Finalement, pour correspondre à $(u_1, u_2) = (g_1, g_2)$ sur Γ_D , on choisit de prendre le vecteur $\vec{u} = (u_1, u_2)$ tel que :

$$\vec{u}|_{\Gamma_D} = \vec{g}$$

En assemblant tout cela, on a :

Trouver $\vec{u} = (u_1, u_2) \in [H^1(\Omega)]^2$ avec $\vec{u}|_{\Gamma_D} = \vec{g}$ et $p \in L^2(\Omega)$ tels que

$$\begin{cases} \nu \int_{\Omega} \nabla \vec{u} : \nabla \vec{v} \, d\Omega - \int_{\Omega} p \operatorname{div}(\vec{v}) \, d\Omega = 0, \quad \forall \vec{v} = (v_1, v_2) \in [H_{0,D}^1(\Omega)]^2 \\ -\int_{\Omega} \operatorname{div}(\vec{u}) q \, d\Omega = 0, \quad \forall q \in L^2(\Omega), \end{cases} \quad (24)$$

où l'on a noté $\nabla \vec{u} : \nabla \vec{v} = \nabla u_1 \cdot \nabla v_1 + \nabla u_2 \cdot \nabla v_2$ et

$$[H_{0,D}^1(\Omega)]^2 = \{\vec{v} \in [H^1(\Omega)]^2 \text{ tel que } \vec{v}|_{\Gamma_D} = (0, 0)\}.$$

Montrons maintenant que réciproquement, cette formulation variationnelle est équivalente au problème de base :

Soit $\varphi \in D(\Omega)$, on a en faisant comme dans l'exercice 1, au sens des distributions :

$$\begin{aligned} & \nu \int_{\Omega} \nabla \vec{u} : \nabla \varphi \, d\Omega - \int_{\Omega} p \operatorname{div}(\varphi) \, d\Omega = 0 \\ \Leftrightarrow & \quad \nu \int_{\Omega} \nabla u_1 \cdot \nabla \varphi_1 + \nabla u_2 \cdot \nabla \varphi_2 \, d\Omega - \int_{\Omega} p \left(\frac{\partial \varphi_1}{\partial x} + \frac{\partial \varphi_2}{\partial y} \right) \, d\Omega = 0 \\ \Leftrightarrow & \quad \langle \nu \nabla u_1 \nabla \varphi_1 \rangle + \langle \nu \nabla u_2 \nabla \varphi_2 \rangle = \left\langle p, \frac{\partial \varphi_1}{\partial x} \right\rangle + \left\langle \frac{\partial \varphi_2}{\partial y} \right\rangle \end{aligned}$$

On a donc, comme $p \in L^2(\Omega) \subset \mathcal{D}'(\Omega)$, et comme $\nabla u_1, \nabla u_2 \in L^2(\Omega)$, $\nabla u_1, \nabla u_2 \in \mathcal{D}'(\Omega)$, on a bien :

$$\langle \nu \Delta u_1, \varphi_1 \rangle + \langle \nu \Delta u_2, \varphi_2 \rangle = \left\langle \frac{\partial p}{\partial x}, \varphi_1 \right\rangle + \left\langle \frac{\partial p}{\partial y}, \varphi_2 \right\rangle$$

On a utilisé les propriétés de la dérivation au sens des distributions et on identifie les termes pour obtenir :

$$\begin{cases} -\nu \Delta u_1 + \frac{\partial p}{\partial x} = 0 & \text{dans } \mathcal{D}'(\Omega) \\ -\nu \Delta u_2 + \frac{\partial p}{\partial y} = 0 & \text{dans } \mathcal{D}'(\Omega) \end{cases}$$

Comme le second membre est dans $L^2(\Omega)$, l'égalité est vraie dans $L^2(\Omega)$ presque partout. Nous en déduisons de plus que $u \in H^1(\Omega, \Delta) \times H^1(\Omega, \Delta)$.

On peut raisonner de la même manière pour la deuxième équation, on a :

$$\begin{aligned} & - \int_{\Omega} \operatorname{div}(\vec{u}) \varphi \, d\Omega = 0 \\ \Leftrightarrow & - \int_{\Omega} \left(\frac{\partial u_1}{\partial x} + \frac{\partial u_2}{\partial y} \right) \varphi \, d\Omega = 0 \\ \Leftrightarrow & \left\langle -\frac{\partial u_1}{\partial x} - \frac{\partial u_2}{\partial y}, \varphi \right\rangle = 0 \end{aligned}$$

On a donc bien :

$$\frac{\partial u_1}{\partial x} + \frac{\partial u_2}{\partial y} = 0 \quad \text{dans } \mathcal{D}'(\Omega), \text{ dans } \Omega, L^2(\Omega) \text{ presque partout}$$

On a directement par l'énoncé :

$$(u_1, u_2) = (g_1, g_2) \quad \text{sur } \Gamma_D$$

Pour les conditions aux limites, on a en choisissant $v \in H^1(\Omega)$ et on peut supposer $u \in H^2(\Omega)$ comme $u \in H^1(\Omega, \Delta)$, on peut appliquer la formule de Green :

$$\begin{aligned} & \nu \int_{\Omega} \nabla \vec{u} : \nabla \varphi \, d\Omega - \int_{\Omega} p \operatorname{div}(\varphi) \, d\Omega = 0 \\ \Leftrightarrow & \nu \int_{\Omega} \nabla u_1 \cdot \nabla \varphi_1 + \nabla u_2 \cdot \nabla \varphi_2 \, d\Omega - \int_{\Omega} p \left(\frac{\partial \varphi_1}{\partial x} + \frac{\partial \varphi_2}{\partial y} \right) \, d\Omega = 0 \\ \Leftrightarrow & \nu \left(\int_{\partial\Omega} (\nabla u_1 \cdot \vec{n}) \varphi_1 \, \partial\Omega - \int_{\Omega} \Delta u_1 \varphi_1 \, d\Omega \right) + \nu \left(\int_{\partial\Omega} (\nabla u_2 \cdot \vec{n}) \varphi_2 \, \partial\Omega - \int_{\Omega} \Delta u_2 \varphi_2 \, d\Omega \right) \\ & - \left(\int_{\partial\Omega} p \varphi_1 n_1 \, \partial\Omega - \int_{\Omega} p \frac{\partial \varphi_1}{\partial x} \, d\Omega + \int_{\partial\Omega} p \varphi_2 n_2 \, \partial\Omega - \int_{\Omega} p \frac{\partial \varphi_2}{\partial y} \, d\Omega \right) = 0 \end{aligned}$$

On rassemble les termes de volume (qui s'annule par le résultat trouvé dans $\mathcal{D}'(\Omega)$) et les termes de bord pour obtenir :

$$\begin{aligned} & \nu \int_{\partial\Omega} (\nabla u_1 \cdot \vec{n}) \varphi_1 \, \partial\Omega + \nu \int_{\partial\Omega} (\nabla u_2 \cdot \vec{n}) \varphi_2 \, \partial\Omega - \int_{\partial\Omega} p \varphi_1 n_1 \, \partial\Omega - \int_{\partial\Omega} p \varphi_2 n_2 \, \partial\Omega = 0 \\ \Leftrightarrow & \int_{\partial\Omega} (\nu (\nabla u_1 \cdot \vec{n}) - p n_1) \varphi_1 \, \partial\Omega + \int_{\partial\Omega} (\nu (\nabla u_2 \cdot \vec{n}) - p n_2) \varphi_2 \, \partial\Omega = 0 \\ \Leftrightarrow & \int_{\partial\Omega} \nu (\nabla \vec{u} \cdot \vec{n}) - p \vec{n} \, \partial\Omega = 0 \end{aligned}$$

On peut écrire :

$$\nu (\nabla \vec{u} \cdot \vec{n}) - p \vec{n} \in \operatorname{Im}(\gamma_0)^\perp$$

Or, $\operatorname{Im}(\gamma_0)$ est dense dans $L^2(\partial\Omega)$, donc $\nu (\nabla \vec{u} \cdot \vec{n}) - p \vec{n} = 0$ presque partout sur $\partial\Omega$, d'où l'équivalence du problème.

On ne peut pas utiliser le théorème de Lax-Milgram, mais on pourrait démontrer d'une autre manière que la formulation variationnelle est bien posée. Comme cela dépasse largement le cadre de ce TP, nous ne détaillerons pas cet aspect.

4.2 Nouveau Maillage

Voici le code du fichier `geomRectangle_partie3.geo` :

```
Mesh.MshFileVersion = 2.2;
// definition du pas du maillage
h = 0.1;
// definition des points (en 3D, raison pour laquelle il y a un 0 en z)
Point(1) = {0, 0, 0, h};
Point(2) = {3, 0, 0, h};
Point(3) = {3, 2, 0, h};
Point(4) = {0, 2, 0, h};
// definition des segments qui relient les points
Line(1) = {1, 2};
Line(2) = {2, 3};
Line(3) = {3, 4};
Line(4) = {4, 1};
// definition des contours fermes
Line Loop(1) = {1,2,3,4};
// definition des surfaces a partir contours fermes
Plane Surface(1) = {1};
// definition des elements physiques : pour ces elements, nous pourrions recuperer
// les references
Physical Point(1) = {1,2,3,4};
Physical Line(1) = {4};
Physical Line(2) = {1,3};
Physical Line(3) = {2};
Physical Surface(1) = {1};
```

Figure 34: Code du fichier `geomRectangle_partie3.geo`

Ce sont les lignes commençant par "Physical", soit les 4 dernières lignes, qui permettront d'imposer les différentes conditions aux bords sur les 4 côtés du rectangle. Les valeurs de *Refneu* permettront d'identifier le bord considéré.

Voici une illustration de ce maillage où rien ne change en représentation visuelle, sauf que pour un même h , on a plus de noeuds dans ce maillage :

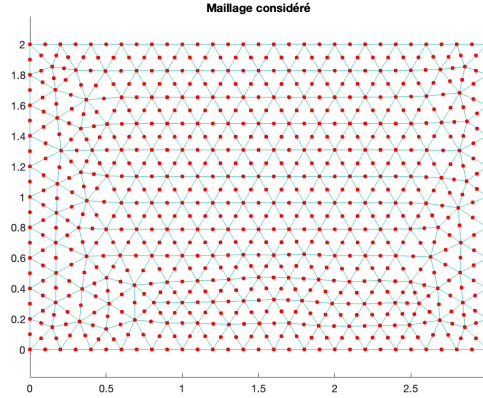


Figure 35: Maillage du partie 3 pour $h = 0.5$

4.3 Discrétisation

Nous discrétisons *chaque composante de la vitesse* par des éléments finis P^2 et la pression par des éléments finis P^1 . Cette paire d'éléments finis est notée $P^2 - P^1$ et s'appelle *élément fini de Taylor-Hood*.

Si on suppose que les sommets des triangles sont numérotés de 1 à N_s et les milieux des arêtes de $N_s + 1$ à N , le vecteur $\vec{U}$ des inconnues du système linéaire issu de la discrétisation est un vecteur de $\mathbb{R}^{2N+N_s}$:

$$\vec{U} = [u_{1_h}(M_1), \dots, u_{1_h}(M_N), u_{2_h}(M_1), \dots, u_{2_h}(M_N), p_h(M_1), \dots, p_h(M_{N_s})]^T,$$

constitué des inconnues relatives à la première composante de la vitesse, puis à la seconde composante de la vitesse, puis à la pression. La matrice du système linéaire peut alors s'écrire par blocs sous la

forme

$$\mathbb{A} = \begin{bmatrix} \nu\mathbb{K} & \mathbf{0} & \mathbb{E} \\ \mathbf{0} & \nu\mathbb{K} & \mathbb{F} \\ \mathbb{G} & \mathbb{H} & \mathbf{0} \end{bmatrix} \in \mathbb{M}_{(2N+N_s)}(\mathbb{R})$$

où

- Les deux blocs $\nu\mathbb{K} \in \mathbb{M}_N(\mathbb{R})$ contiennent les contributions issues de la discrétisation des termes $\nu \int_{\Omega} \nabla u_1 \cdot \nabla v_1 \, d\Omega$ et $\nu \int_{\Omega} \nabla u_2 \cdot \nabla v_2 \, d\Omega$. En d'autres termes, on a

$$\mathbb{K}_{IJ} = \int_{\Omega} \nabla w_I \cdot \nabla w_J \, d\Omega$$

avec $(w_I)_{I=1,N}$ les fonctions de base P^2 utilisées dans les parties précédentes.

- Le bloc rectangulaire $\mathbb{E} \in \mathbb{M}_{N,N_s}(\mathbb{R})$ contient les contributions issues de la discrétisation du terme $-\int_{\Omega} p \frac{\partial v_1}{\partial x} \, d\Omega$, soit

$$\mathbb{E}_{IJ} = - \int_{\Omega} \tilde{w}_J \frac{\partial w_I}{\partial x} \, d\Omega$$

avec $(\tilde{w}_J)_{J=1,N_s}$ les fonctions de base P^1 utilisées dans le TP1, soit tel que $w_J(M_I) = \delta_{IJ}$, $\forall I, J \in [1, N] \times [1, N]$.

- Le bloc rectangulaire $\mathbb{F} \in \mathbb{M}_{N,N_s}(\mathbb{R})$ contient les contributions issues de la discrétisation du terme $-\int_{\Omega} p \frac{\partial v_2}{\partial y} \, d\Omega$, à savoir

$$\mathbb{F}_{IJ} = - \int_{\Omega} \tilde{w}_J \frac{\partial w_I}{\partial y} \, d\Omega$$

Comme :

$$\int_{\Omega} \operatorname{div}(\vec{u}) \, q \, d\Omega = 0, \quad \forall q \in L^2(\Omega),$$

On peut en particulier prendre $q = p$ et on a :

$$\int_{\Omega} \left(\frac{\partial v_1}{\partial x} + \frac{\partial v_2}{\partial y} \right) p \, d\Omega = 0$$

L'échange de v_1 et v_2 dans l'équation correspond à l'opération de transposition. Les blocs $\mathbb{G}$ et $\mathbb{H}$, aussi issus de la discrétisation des termes de la pression sont tels que : $\mathbb{G} = \mathbb{E}^T$, $\mathbb{H} = \mathbb{F}^T$.

4.4 Calcul des matrices élémentaires

On écrit alors deux routines `matE_elem.m` et `matF_elem.m` qui à partir des coordonnées des 3 sommets d'un triangle T_ℓ donnent respectivement les matrices élémentaires $\mathbb{E}^\ell \in \mathbb{M}_{6,3}(\mathbb{R})$ et $\mathbb{F}^\ell \in \mathbb{M}_{6,3}(\mathbb{R})$. Pour ce faire, on remarque que $\tilde{w}_J|_{T_\ell} = \lambda_J$ et on utilise les formules suivantes obtenues par passage à l'élément fini de référence : pour $I = 1, \dots, 6$, $J = 1, \dots, 3$, le coefficient $(\mathbb{E}^\ell)_{IJ}$ peut s'écrire :

$$- \int_{T_\ell} \lambda_J(M) \frac{\partial w_I(M)}{\partial x} \, d\Omega = - \int_{\hat{T}} \hat{\lambda}_J(\hat{M}) \operatorname{ligne}_1\{(\mathcal{B}_\ell^T)^{-1}\} \hat{\nabla} \hat{w}_I(\hat{M}) |\det \mathcal{B}_\ell| \, d\hat{\Omega},$$

et le coefficient $(\mathbb{F}^\ell)_{IJ}$ peut s'écrire :

$$- \int_{T_\ell} \lambda_J(M) \frac{\partial w_I(M)}{\partial y} \, d\Omega = - \int_{\hat{T}} \hat{\lambda}_J(\hat{M}) \operatorname{ligne}_2\{(\mathcal{B}_\ell^T)^{-1}\} \hat{\nabla} \hat{w}_I(\hat{M}) |\det \mathcal{B}_\ell| \, d\hat{\Omega},$$

où l'on a utilisé la notation $\operatorname{ligne}_m\{\mathbb{B}\}$ pour indiquer la m -ième ligne d'une matrice $\mathbb{B}$. Pour calculer les intégrales sur $\hat{T}$ ci-dessus, on utilise une formule de quadrature d'ordre 2, car l'expression de ces matrices se rapprochent bien de celles des matrices élémentaires de rigidité calculées précédemment. Voici les codes de ces deux routines, en prenant les mêmes notations qu'avant pour $B\ell$, D , $\dots$:

```

% On se place sur le triangle de référence :
% -----
x1 = 0;
y1 = 0;
x2 = 1;
y2 = 0;
x3 = 0;
y3 = 1;

% Points et poids de quadrature (ordre 2)
% -----
S_hat = [1/6, 1/6;
         2/3, 1/6;
         1/6, 2/3];
poids = 1/6;

% Calcul de la matrice elementaire du bloc rectangulaire (p, dv1/dx)
% -----
Eel = zeros(6,3);

% Calcul des contributions via quadrature
% -----
for i=1:6
    for j=1:6
        G= 0;
        for k=1:3
            G = G - poids * abs(D) * lambda(j, S_hat(k,1), S_hat(k,2), x1, y1, x2, y2, x3, y3) * Bl_invtranspose(1, :) * grad_w(i, S_hat(k,1), S_
        end
        Eel(i,j) = G;
    end
end

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%                                     fin de la routine
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%24

```

Figure 36: Code de la routine `matE_elem.m`

```

% On se place sur le triangle de référence :
% -----
x1 = 0;
y1 = 0;
x2 = 1;
y2 = 0;
x3 = 0;
y3 = 1;

% Points et poids de quadrature (ordre 2)
% -----
S_hat = [1/6, 1/6;
         2/3, 1/6;
         1/6, 2/3];
poids = 1/6;

% calcul de la matrice elementaire du bloc rectangulaire (p, dv2/dy)
% -----
Fel = zeros(6,3);

% Calcul des contributions via quadrature
% -----
for i=1:6
    for j=1:6
        G= 0;
        for k=1:3
            G = G - poids * abs(D) * lambda(j, S_hat(k,1), S_hat(k,2), x1, y1, x2, y2, x3, y3) * Bl_invtranspose(2, :) * grad_w(i, S_hat(k,1), S_
        end
        Fel(i,j) = G;
    end
end

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%                                     fin de la routine
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%24

```

Figure 37: Code de la routine `matF_elem.m`

4.5 Assemblage des matrices

Voici le code pour l'assemblage des matrices, où on initialise les matrices, les assemble, réduit selon les sommets et non plus selon les sommets et les arêtes, et on assemble à nouveau le tout :

```

% -----
% calcul des matrices EF
% -----
% declarations
% -----
% on ne connait pas encore le nombre de sommets, attention ce n'est pas la moitie du nombre des noeuds !
MM = sparse(Nbpt, Nbpt); % matrice de masse
KK = sparse(Nbpt, Nbpt); % matrice de rigidite : bloc (u_i, v_i)
EEtmp = zeros(Nbpt, Nbpt); % bloc (p, v1) carre temporaire, ce bloc sera rectangulaire apres extraction des noeuds correspondant aux sommets
FFtmp = zeros(Nbpt, Nbpt); % bloc (p, v2) carre temporaire, ce bloc sera rectangulaire apres extraction des noeuds correspondant aux sommets
isavertex = zeros(Nbpt, 1); % tableau boolean "is a vertex" reperant les noeuds qui sont des sommets (et pas des milieux)

```

Figure 38: Initialisation des matrices du fichier principal

```

for l=1:Nbtri

    % Coordonnees des sommets du triangle
    T = Numtri(l,:);
    S1=Coorneu(T(1,:),);
    S2=Coorneu(T(2,:),);
    S3=Coorneu(T(3,:),);

    % On met 1 dans les entrees de isavertex correspondants aux 3 sommets
    isavertex(Numtri(l,1)) = 1;
    isavertex(Numtri(l,2)) = 1;
    isavertex(Numtri(l,3)) = 1;

    % Calcul des matrices elementaires du triangle l
    Mel = matM_elem_p2(S1, S2, S3);
    Kel = matK_elem_p2(S1, S2, S3);
    Eel = matE_elem(S1, S2, S3);
    Fel = matF_elem(S1, S2, S3);

    % On fait l assemblage des blocs (u_i, v_i)
    for i=1:6
        I = T(i);
        for j=1:6
            J = T(j);
            KK(I,J) = KK(I,J) + Kel(i,j);
        end
    end

    % On fait l assemblage des blocs (p, v1) et (p, v2)
    for i=1:6
        I = T(i);
        for j=1:3
            J = T(j);
            EEtmp(I,J) = EEtmp(I,J) + Eel(i,j);
            FFtmp(I,J) = FFtmp(I,J) + Fel(i,j);
        end
    end

end % for l

```

Figure 39: Assemblage des différentes matrices par blocs

```

% find(isavertex~=0) donne la liste des numeros des sommets
numssommets = find(isavertex==1);
Ns = length(numssommets); % nombre de sommets

EE = zeros(Nbpt, Ns); % bloc rectangulaire (p, v1)
FF = zeros(Nbpt, Ns); % bloc rectangulaire (p, v2)

% On extrait de EEtmp et FFtmp les colonnes correspondantes a des sommets
% (par ailleurs les autres colonnes sont a priori nulles)
EE=EEtmp(:,numssommets);
FF=FFtmp(:,numssommets);

```

Figure 40: Réduction des blocs liés aux termes de pressions seulement aux sommets

```

% Matrice elements finis par blocs
% -----
AA = sparse(2*Nbpt + Ns, 2*Nbpt + Ns);

% A COMPLETER
AA(1:Nbpt, 1:Nbpt) = nu * KK;
AA((Nbpt+1):(2*Nbpt), (Nbpt+1):(2*Nbpt)) = nu * KK;
AA(1:Nbpt, (2*Nbpt+1):(2*Nbpt + Ns)) = EE;
AA((Nbpt+1):(2*Nbpt), (2*Nbpt+1):(2*Nbpt + Ns)) = FF;
AA((2*Nbpt+1):(2*Nbpt + Ns), 1:Nbpt) = EE';
AA((2*Nbpt+1):(2*Nbpt + Ns), (Nbpt+1):(2*Nbpt)) = FF';

```

Figure 41: Remplissage final de la matrice A

4.6 Second membre et pseudo-élimination

On écrit la routine `elimine_stokes.m` pour imposer par pseudo-élimination des conditions de Dirichlet sur une partie Γ_D du bord (*pour chacune des deux composantes de la vitesse*), comme dans l'équation de Stokes, dont voici les codes :

```
function [AA_tilde, LL_tilde] = elimine_stokes(AA, LL, Refneu, Coorneu)

AA_tilde = AA;
LL_tilde = LL;
Nbpt=size(Refneu,1);
for i=1:Nbpt
    % si on est sur la partie du bord concernée par les conditions aux limites de Dirichlet
    % côté gauche, et les deux horizontaux
    if Refneu(i)~=0 && (Coorneu(i,2)==0 || Coorneu(i,2)==2 || Coorneu(i,1)==0)
        AA_tilde(i,:)=0;
        AA_tilde(i+Nbpt,:)=0;
        AA_tilde(i,i)=1;
        AA_tilde(i+Nbpt,i+Nbpt)=1;
        LL_tilde(i)=g1(Refneu, Coorneu, i);
        LL_tilde(i+Nbpt)=g2(Refneu, Coorneu, i);
    end
end
end
```

Figure 42: Code de la routine `elimine_stokes.m`

On injecte cela dans le fichier principal pour résoudre le problème de Stokes (*Ici, on inclue déjà l'option d'effectuer la partie marche dans le code*) :

```
% Pseudo-elimination
% -----
% on n'impose Dirichlet que sur les inconnues P2 de la vitesse
LL = zeros(2*Nbpt+Ns,1); % vecteur second membre
tilde_AA = sparse(2*Nbpt + Ns, 2*Nbpt + Ns);
tilde_LL = zeros(2*Nbpt+Ns,1);
if is_marche == 0
    [tilde_AA, tilde_LL] = elimine_stokes(AA,LL,Refneu,Coorneu);
elseif is_marche == 1
    [tilde_AA, tilde_LL] = elimine_stokes_marche(AA,LL,Refneu,Coorneu);
end

% Resolution du systeme lineaire
% -----
UU = tilde_AA\tilde_LL;
```

Figure 43: Résolution dans le fichier principal après pseudo-élimination

On a ensuite le fichier principal qui résout le problème, il possède différents paramètres à prendre en compte avant de lancer, que voici :

```
% nu = le coefficient de viscosité (1 pour le début la partie 3, etc)
% h_wanted = la taille de maillage voulue
% is_marche = 1 si on veut faire le problème de la marche, 0 sinon
% validation = 'oui' si on veut regarder les erreurs, 'non' sinon
% init_visualisation = 1 si on veut voir le maillage de départ, 0 sinon
% result_visualisation = 1 si on veut voir le résultat : en approché et en Exact
% vect_vitesse_visualisation = 1 si on veut voir les vecteurs vitesse avec les flèches
% erreur_visualisation = 1 si on veut voir les graphes des erreurs
function [Diff_U1, Diff_U2, Diff_P] = principal_stokes(nu, h_wanted, is_marche, validation,
    init_visualisation, result_visualisation, vect_vitesse_visualisation, erreur_visualisation)
```

Figure 44: Template du fichier principal `principal_stokes.m`

4.7 Validation avec un exemple

On veut vérifier que le code calcule une solution approchée (u_{1h}, u_{2h}, p_h) correcte. Pour cela, on résout le problème avec le domaine rectangulaire $\Omega =]0, 3[\times]0, 2[$, une viscosité $\nu = 1$.

4.7.1 Calcul de $\bar{g}$

- des conditions de Dirichlet homogènes (c'est-à-dire $(g_1, g_2) = (0, 0)$) sur les deux côtés horizontaux du rectangle ;

- des conditions de Dirichlet non homogènes avec $(g_1, g_2) = ((2 - y)y, 0)$ sur le côté gauche du rectangle, pour imposer un profil d'écoulement entrant parabolique ;
- des conditions de Neumann homogènes comme dans en sortie, sur le côté vertical à droite du rectangle.

On a alors les codes suivants pour les routines `g1.m` et `g2.m` :

```
function g1_value = g1(Refneu, Coorneu, i)

if Refneu(i)~=0
    % sur les côtés horizontaux
    if Coorneu(i,2)==0 || Coorneu(i,2)==2
        g1_value=0;
    end
    % sur le côté gauche du rectangle
    if Coorneu(i,1)==0
        g1_value=(2-Coorneu(i,2))*Coorneu(i,2);
    end
end
end
```

(a) Code de la routine `g1.m`

```
function g2_value = g2(Refneu, Coorneu, i)

if Refneu(i)~=0
    % nul sur TD
    if Coorneu(i,2)==0 || Coorneu(i,2)==2 || Coorneu(i,1)==0
        g2_value=0;
    end
end
end
```

(b) Code de la routine `g2.m`

Figure 45: Codes des routines `g1.m` et `g2.m`

4.7.2 Solution exacte, vérification

On a $(u_1, u_2) = ((2 - y)y, 0)$ et $p = -2(x - 3)$, comme solution exacte. Vérifions d'abord que ces fonctions sont bien solution du problème :

- $-\nu \Delta u_1 + \frac{\partial p}{\partial x} = -(-2) + (-2) = 0$
- $-\nu \Delta u_2 + \frac{\partial p}{\partial y} = 0 + 0 = 0.$
- $\text{div}(\vec{u}) = \frac{\partial u_1}{\partial x} + \frac{\partial u_2}{\partial y} = 0.$
- On a $(u_1, u_2) = (g_1, g_2)$ sur Γ_D , que l'on décompose sur $y = 0$ (*en bas*), $y = 2$ (*en haut*) et $x = 0$ (*à gauche*) :

Sur $\Gamma_{D_{gauche}}$, $u_1 = (2 - y)y = g_1$ et $u_2 = 0 = g_2$.

Sur $\Gamma_{D_{bas}}$, $u_1 = 0 = g_1$ et $u_2 = 0 = g_2$ car $y = 0$.

Sur $\Gamma_{D_{haut}}$, $u_1 = 0 = g_1$ et $u_2 = 0 = g_2$ car $y = 2$, $2 - y = 0$.

- On a $\nu \nabla u_1 \cdot \vec{n} - pn_1 = 0$ et $\nu \nabla u_2 \cdot \vec{n} - pn_2 = 0$ sur Γ_N (*à droite*, $x = 3$).
- $\nu \nabla u_1 \cdot \vec{n} - pn_1 = \frac{\partial u_1}{\partial x} n_1 + \frac{\partial u_1}{\partial y} n_2 - pn_1 = 0 + (2 - 2y)n_2 + 2(x - 3)n_1 = 0$ car $x = 3$ et $n_2 = 0$ sur Γ_N .
- $\nu \nabla u_2 \cdot \vec{n} - pn_2 = \frac{\partial u_2}{\partial x} n_1 + \frac{\partial u_2}{\partial y} n_2 - pn_2 = -pn_2 = 0$ car $n_2 = 0$ sur Γ_N .

4.7.3 Premier résultats

Passons maintenant à la résolution du problème, en affichant les différents résultats obtenus pour $h = 0.5$, voici divers graphiques obtenus, avec d'abord les codes associés :

```

% visualisation solution approchée
% -----
U1 = UU(1:Nbpt);
U2 = UU((Nbpt+1):(2*Nbpt));
P = UU((2*Nbpt+1):(2*Nbpt + Ns));

if result_visualisation == 1
    affiche_ordre2(U1, Numtri, Coorneu,sprintf('U1 - %s', nom_maillage));
    affiche_ordre2(U2, Numtri, Coorneu,sprintf('U2 - %s', nom_maillage));
end

% Dans la i-eme entree de DEnumnoeudAnumsommet on trouve le numero en tant que sommet du i-eme noe
% (et on trouve 0 si l'i-eme noeud n'est pas un sommet)
DEnumnoeudAnumsommet = zeros(Nbpt, 1);
DEnumnoeudAnumsommet(numsommets) = 1:Ns;
% Pour l'affichage on decale les numeros dans Numtri
NumtriDecale = zeros(Nbtri,3);
NumtriDecale = DEnumnoeudAnumsommet(Numtri(:,1:3));
if result_visualisation == 1
    affiche_ordre1(P, NumtriDecale, Coorneu(numsommets,:),sprintf('pression - %s', nom_maillage));
end

% Affichage des flèches du vecteur vitesse
if vect_vitesse_visualisation == 1
    figure;
    hold on;
    quiver(Coorneu(:,1), Coorneu(:,2), U1, U2, 'r');
    title(sprintf('Champ de vitesse - %s', nom_maillage));
    xlabel('x', 'FontSize', 12);
    ylabel('y', 'FontSize', 12);
    axis equal;
    grid on;
    hold off;
end

```

Figure 46: Visualisation des résultats approchés

```

% validation seulement si voulu
% -----
% On initialise quand même les erreurs :
% On n'a pas besoin de cette partie pour la marche
Diff_U1 = zeros(Nbpt, 1);
Diff_U2 = zeros(Nbpt, 1);
Diff_P = zeros(Ns, 1);
if strcmp(validation,'oui')

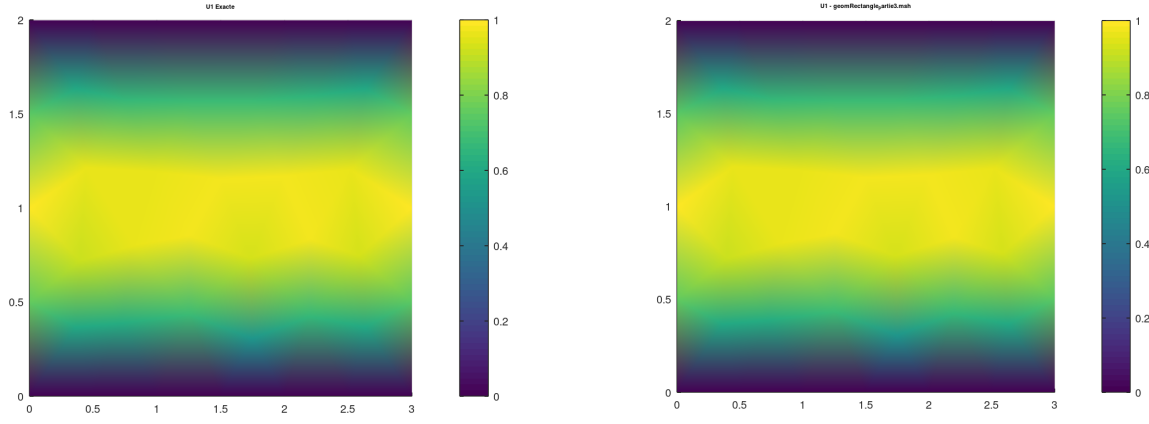
    % Solutions exactes
    U1_exact = (2-Coorneu(:,2)) .* Coorneu(:,2);
    U2_exact = zeros(size(Coorneu, 1), 1);
    P_exact = -2*(Coorneu(numsommets,1)-3) ;
    if result_visualisation == 1
        affiche_ordre2(U1_exact, Numtri, Coorneu, "U1 Exacte");
        affiche_ordre2(U2_exact, Numtri, Coorneu, "U2 Exacte");
        affiche_ordre1(P_exact, NumtriDecale, Coorneu(numsommets,:), "Pression Exacte");
    end

    % Calcul des différences entre solution exacte et approchée
    Diff_U1 = U1_exact - U1;
    Diff_U2 = U2_exact - U2;
    Diff_P = P_exact - P;

    if erreur_visualisation == 1
        affiche_ordre2(Diff_U1, Numtri, Coorneu, 'Différence u1 - u1h');
        affiche_ordre2(Diff_U2, Numtri, Coorneu, 'Différence u2 - u2h');
        affiche_ordre1(Diff_P, NumtriDecale, Coorneu(numsommets,:), 'Différence p - ph');
    end
end

```

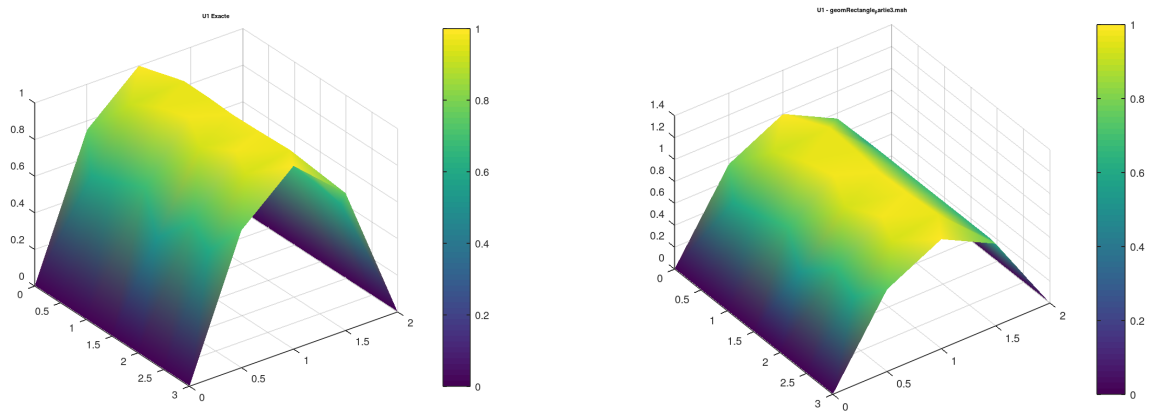
Figure 47: Code de calcul et d'affichage des solutions exactes et des erreurs



(a) Solution exacte

(b) Solution approchée

Figure 48: Résultats 2D pour u_1 pour $h = 0.5$

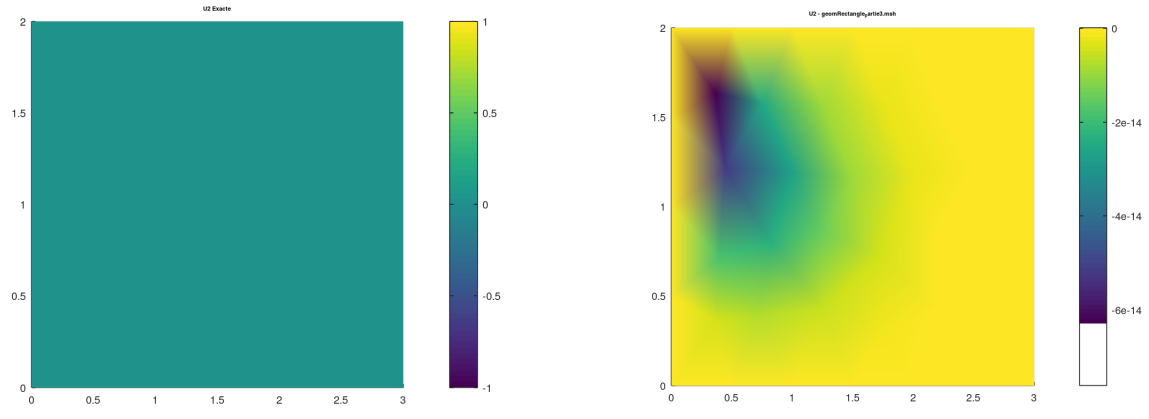


(a) Solution exacte

(b) Solution approchée

Figure 49: Résultats 3D pour u_1 pour $h = 0.5$

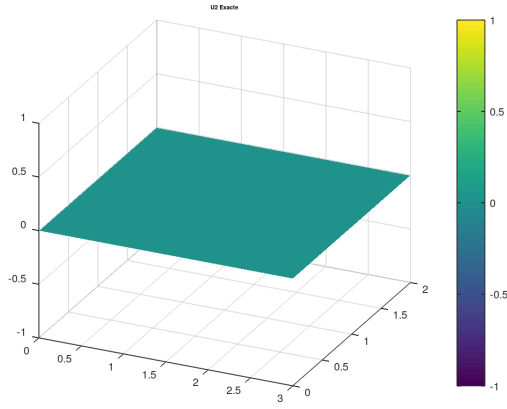
Les solutions exactes et approchées sont quasiment similaires de loin, même si l'on voit quelques différences très minimes lorsque l'on regarde de plus près



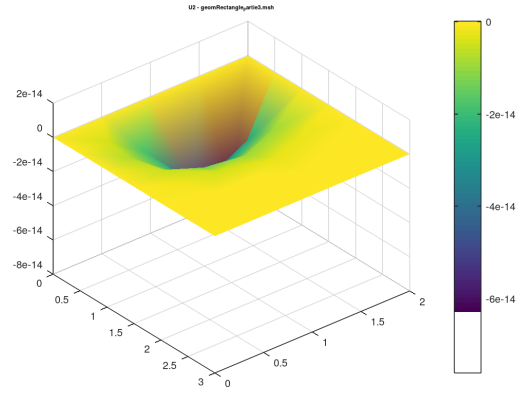
(a) Solution exacte

(b) Solution approchée

Figure 50: Résultats 2D pour u_2 pour $h = 0.5$



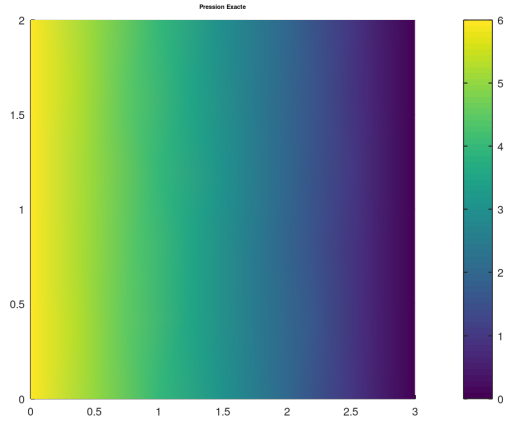
(a) Solution exacte



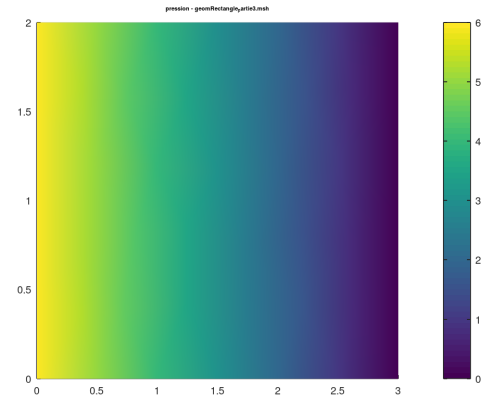
(b) Solution approchée

Figure 51: Résultats 3D pour u_2 pour $h = 0.5$

On peut clairement voir que la solution approchée est très proche de la solution exacte pour u_2 en regardant les ordre de grandeur des valeurs en ordonnée des graphiques.

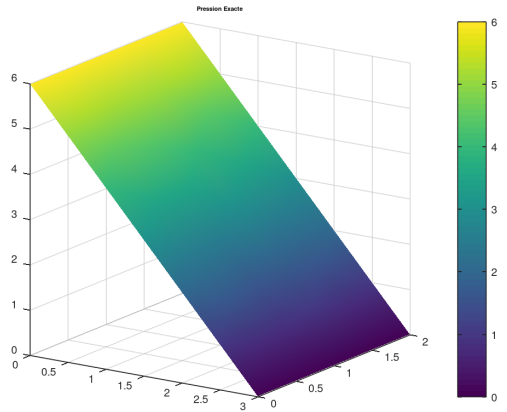


(a) Solution exacte

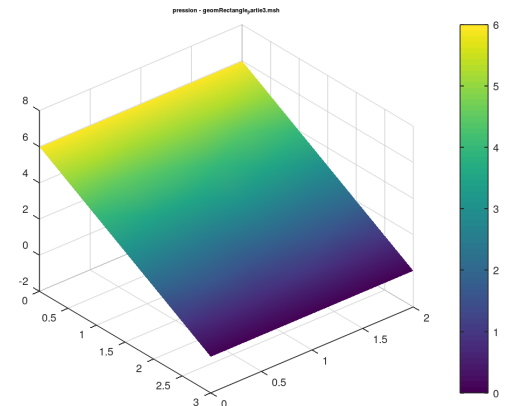


(b) Solution approchée

Figure 52: Résultats 2D pour p pour $h = 0.5$



(a) Solution exacte



(b) Solution approchée

Figure 53: Résultats 3D pour p pour $h = 0.5$

Globalement, la pression suit la même tendance, elle diminue au fur et au mesure qu'on avance dans le maillage, avec les mêmes dynamiques et ordre de grandeur. On peut de plus ajouter que les ordres de grandeur sont très faibles par rapport à la solution exacte. On peut aussi afficher les vecteurs vitesse pour se rendre compte de la dynamique de l'écoulement :

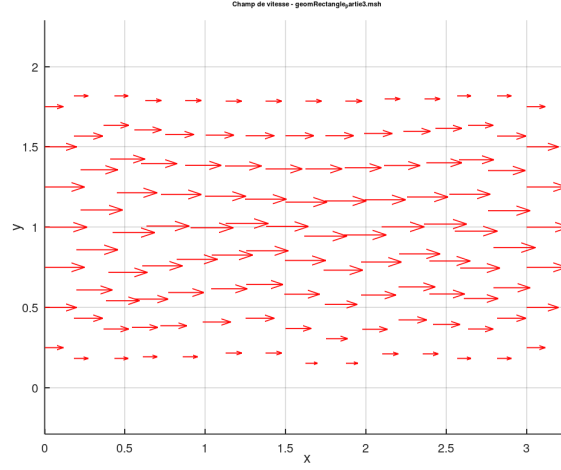


Figure 54: Vecteurs vitesse pour $h = 0.5$

On peut voir que les vecteurs vitesse sont bien orientés, l'écoulement se fait le long de la conduite et est bien dirigé vers la droite, avec une vitesse restant plutôt constante le long de la conduite. On voit aussi que la vitesse est bien moindre au niveau des bords de la conduite.

Ensuite on représente, dans trois graphiques séparés, les différences $u_1 - u_{1h}$, $u_2 - u_{2h}$ et $p - p_h$, pour comparer chaque composante de la solution exacte et de la solution approchée :

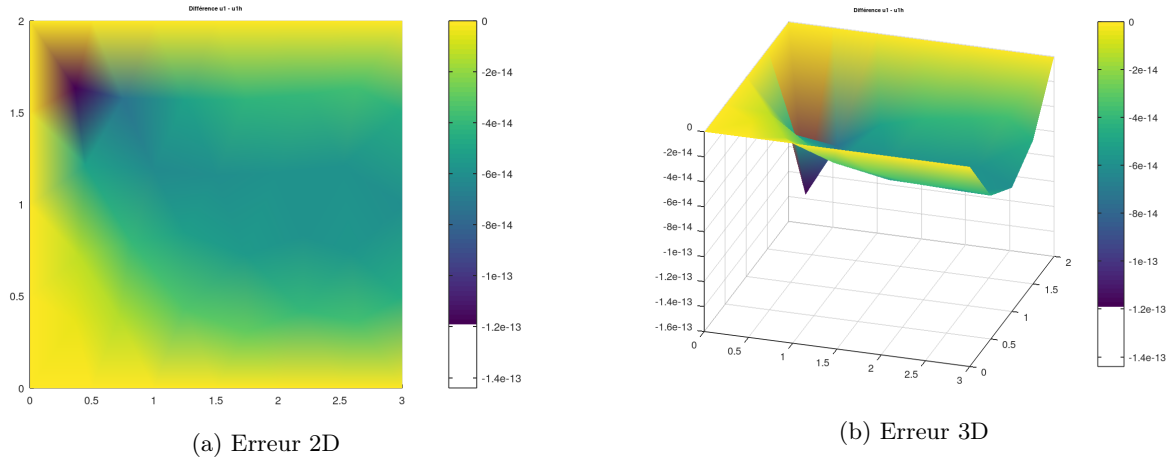


Figure 55: Différences $u_1 - u_{1h}$ pour $h = 0.5$

Le graphique de la différence représente bien les faibles différences entre la solution exacte et la solution approchée.

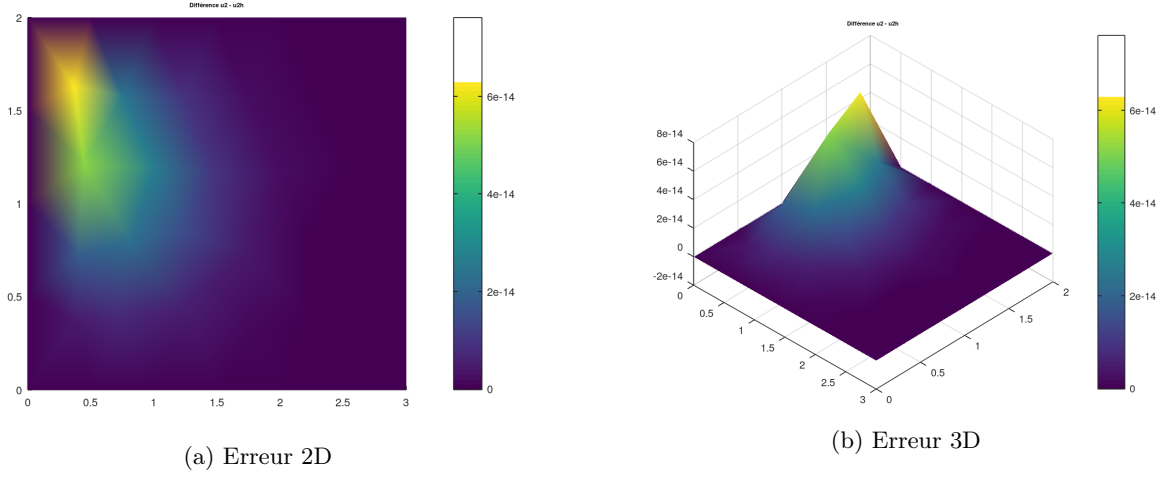


Figure 56: Différences $u_2 - u_{2h}$ pour $h = 0.5$

Encore une fois, ici les erreurs sont minimales car la solution approchée est quasiment nulle, tandis que la solution exacte est nulle.

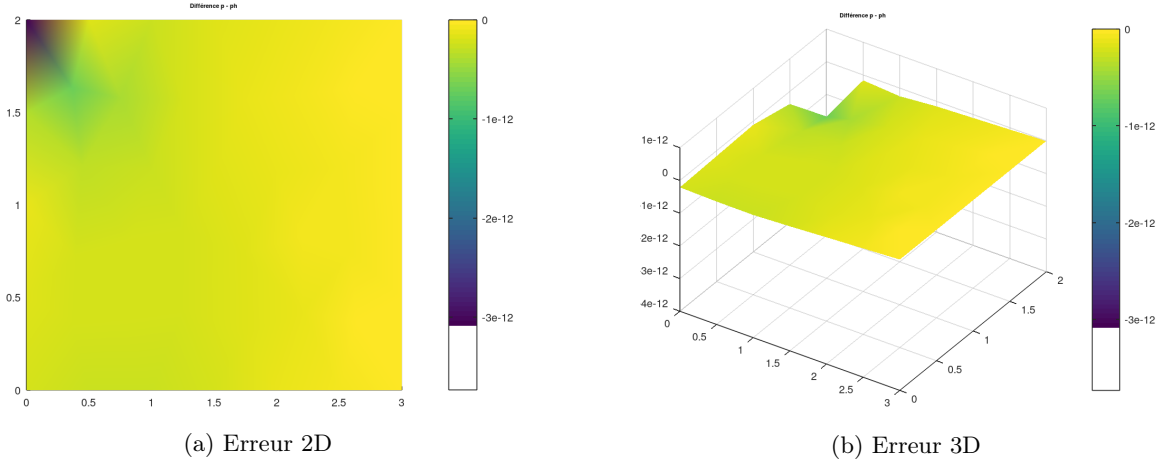


Figure 57: Différences $p - p_h$ pour $h = 0.5$

Les erreurs pour la pression sont là aussi très petites.

4.7.4 Analyse des résultats

Traçons maintenant les erreurs pour différentes tailles de maillage, voici les codes pour se faire, les régressions linéaires en échelle $\log - \log$ sont les mêmes que précédemment :

```
% Les 4 tailles de maillage que je souhaite tester
h = [0.5, 0.2, 0.1, 0.05];
Diff_U1 = [0, 0, 0, 0];
Diff_U2 = [0, 0, 0, 0];
Diff_P = [0, 0, 0, 0];
for i=1:4
    [Diff_U1_tested, Diff_U2_tested, Diff_P_tested] = principal_stokes(1, h(i), 0, 'oui', 0, 0, 0, 0);
    Diff_U1(i) = sqrt(sum(Diff_U1_tested.^2));
    Diff_U2(i) = sqrt(sum(Diff_U2_tested.^2));
    Diff_P(i) = sqrt(sum(Diff_P_tested.^2));
end
```

Figure 58: Code de calcul des erreurs pour différentes tailles de maillage

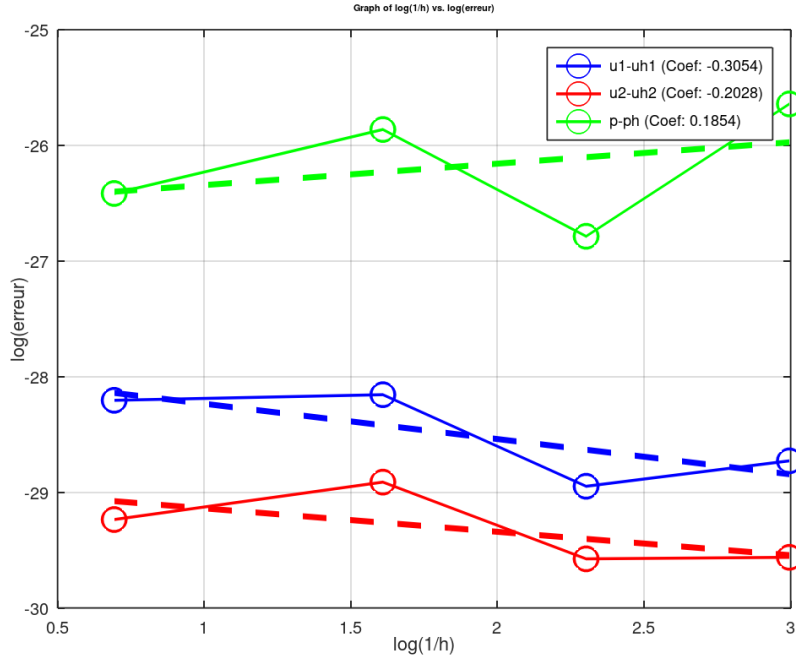


Figure 59: Erreurs pour différentes tailles de maillage

Les erreurs en norme L^2 sont similaires entre les différentes tailles de maillage, tout comme les graphiques des solutions à quelques détails près. On peut voir qu'il y a peu d'erreurs grâce aux faible échelle d'ordonnée du graphique.

4.8 Écoulement d'un fluide dans un canal avec une marche descendante

On s'intéresse maintenant à l'écoulement d'un fluide dans un canal avec une marche descendante dont voici la figure :

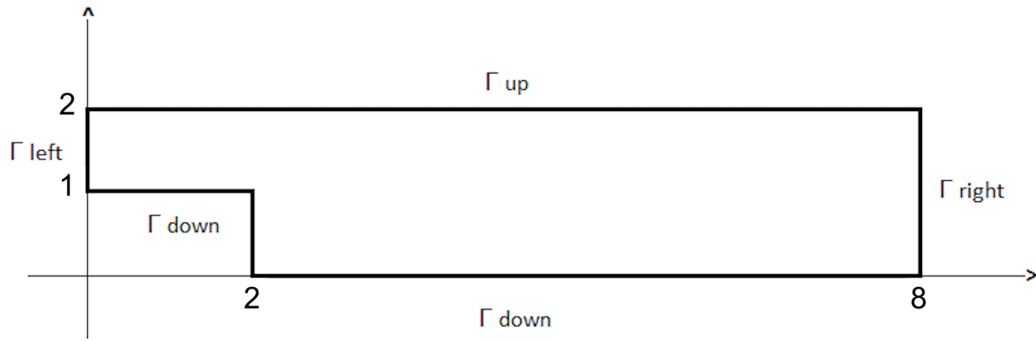


Figure 60: Domaine de calcul pour un canal avec une marche descendante

On notera Γ_{left} le segment du bord à gauche avec $1 \leq y \leq 2$, Γ_{right} le bord de droite, Γ_{up} le bord en haut et Γ_{down} le reste du bord. On impose sur Γ_{left} un profil d'écoulement entrant parabolique de vitesse maximale *unitaire* et qui vaut 0 en $(0, 1)$ et en $(0, 2)$, une vitesse nulle sur Γ_{up} et Γ_{down} , et des conditions de Neumann homogènes en sortie sur Γ_{right} .

4.8.1 Modification du maillage

Pour prendre en compte la marche, on modifie le fichier `geomRectangle_partie3_marche.geo` pour ajouter la marche descendante, dont voici le code :

```

Mesh.MshFileVersion = 2.2;
// definition du pas du maillage
h = 0.50;
// definition des points (en 3D, raison pour laquelle il y a un 0 en z)
Point(1) = {2, 0, 0, h};
Point(2) = {8, 0, 0, h};
Point(3) = {8, 2, 0, h};
Point(4) = {0, 2, 0, h};
Point(5) = {0, 1, 0, h};
Point(6) = {2, 1, 0, h};
// definition des segments qui relient les points
Line(1) = {1, 2};
Line(2) = {2, 3};
Line(3) = {3, 4};
Line(4) = {4, 5};
Line(5) = {5, 6};
Line(6) = {6, 1};
// definition des contours fermes
Line Loop(1) = {1,2,3,4,5,6};
// definition des surfaces a partir contours fermes
Plane Surface(1) = {1};
// definition des elements physiques : pour ces elements, nous pourrions recuperer
// les references
Physical Point(1) = {1,2,3,4,5,6};
Physical Line(1) = {1,6,5};
Physical Line(2) = {4};
Physical Line(3) = {3};
Physical Line(4) = {2};
Physical Surface(1) = {1};

```

Figure 61: Code du fichier `geomRectangle_partie3_marche.geo`

On doit changer le main ensuite, en incluant une nouvelle fonction pour automatiser le changement de la taille du maillage, comme fait précédemment :

```

function h_change_maillage_stokes_marche(new_h_value)
% Lire le fichier .geo
filename = 'geomRectangle_partie3_marche.geo';
fileContents = fileread(filename);

% Remplacer la valeur de h par la nouvelle valeur souhaitée
pattern = 'h = [0-9]*\.[0-9]*'; % Correspond à la ligne définissant h
replacement = sprintf('h = %.2f', new_h_value); % Remplace par la nouvelle valeur de h

% Modifier le contenu du fichier
newFileContents = regexprep(fileContents, pattern, replacement);

% Écrire les modifications dans le fichier
fid = fopen(filename, 'w');
fwrite(fid, newFileContents);
fclose(fid);

% Exécuter Gmsh pour générer le maillage du geo au msh
% J'ai téléchargé par homebrew, le path est à changer selon la localisation
% de l'outil dans le système
system('/opt/homebrew/bin/gmsh geomRectangle_partie3_marche.geo -2 -order 2 -o geomRectangle_partie3_
end

```

Figure 62: Code du fichier `h_change_maillage_stokes_marche.m`

```

% Changement de la valeur de h avant de calculer le tout
% -----
if is_marche == 0
    h_change_maillage_stokes(h_wanted);
elseif is_marche == 1
    h_change_maillage_stokes_marche(h_wanted);
end

% lecture du maillage et affichage
% -----
nom_maillage = 'geomRectangle_partie3.msh';
% si on est sur le cas de la marche, il faut prendre le maillage de la marche
if is_marche == 1
    nom_maillage = 'geomRectangle_partie3_marche.msh';
endif
[Nbpt,Nbtri,Coorneu,Refneu,Numtri,Reftri,Nbaretes,Numaretes,Refaretes] = lecture_msh_ordre2(nom_maillage);
if init_visualisation == 1
    affichage_maillage_ordre2(nom_maillage, "Maillage considéré");
end

```

Figure 63: Changement dans le fichier principal inclure le maillage de la marche

On obtient alors le maillage suivant :

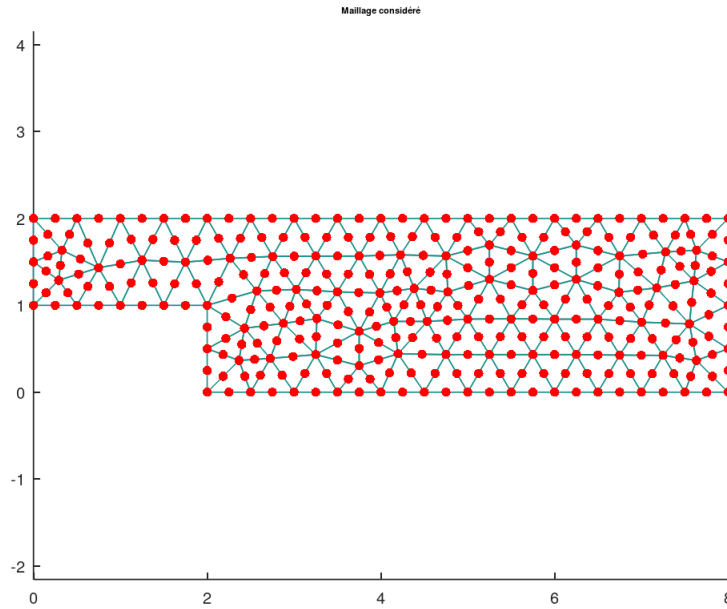


Figure 64: Maillage de la marche descendante pour $h = 0.5$

4.8.2 Modification des routines de pseudo-élimination et de second membre

On doit modifier les routines de pseudo-élimination et de second membre pour prendre en compte la marche descendante, dont voici les codes :

```
function g1_value = g1_marche(Refneu, Coorneu, i)

    g1_value=0;
    % Bord où les conditions de Dirichlet sont appliquées
    % on enlève les points (0,1) et (0,2) ici
    if Coorneu(i,1)==0 && 1<Coorneu(i,2)<2
        g1_value=(2-Coorneu(i,2))*Coorneu(i,2);
    end
end
```

(a) Code de la routine `g1_marche.m`

```
function g2_value = g2_marche(Refneu, Coorneu, i)
    g2_value = 0;
end
```

(b) Code de la routine `g2_marche.m`

Figure 65: Codes des routines `g1_marche.m` et `g2_marche.m`

```
function [AA_tilde, LL_tilde] = elimine_stokes_marche(AA, LL, Refneu, Coorneu)

    AA_tilde = AA;
    LL_tilde = LL;
    size_mat = size(AA_tilde,1);
    Nbpt=size(Refneu,1);
    for i=1:Nbpt
        % si on est sur la partie du bord concernée par les conditions aux limites de Dirichlet
        % Tdown(x de 2 à 8), Tdown(x=2, y<1), Tdown(y=1), Tdown(x=0, y>1), Tup(y=2)
        if Refneu(i)==0 && ((Coorneu(i,1)>2 && Coorneu(i,2)==0) || (Coorneu(i,1)==2 && Coorneu(i,2)<1)
            || (0<Coorneu(i,1)<2 && Coorneu(i,2)==1) || (Coorneu(i,1)==0 && 1<Coorneu(i,2)<2) || Coorneu(i,2)==2)
            AA_tilde(i,:)=0;
            AA_tilde(i+Nbpt,:)=0;
            AA_tilde(i,i)=1;
            AA_tilde(i+Nbpt,i+Nbpt)=1;
            LL_tilde(i)=g1_marche(Refneu, Coorneu, i);
            LL_tilde(i+Nbpt)=g2_marche(Refneu, Coorneu, i);
        end
    end
end
```

Figure 66: Code de la routine `elimine_stokes_marche.m`

4.8.3 Validation

Affichons les résultats pour la marche descendante pour $\nu = 1$ et $h = 0.5$, avec les mêmes codes que précédemment pour le fichier principal et l'intégration des quelques modifications liées à cette partie :

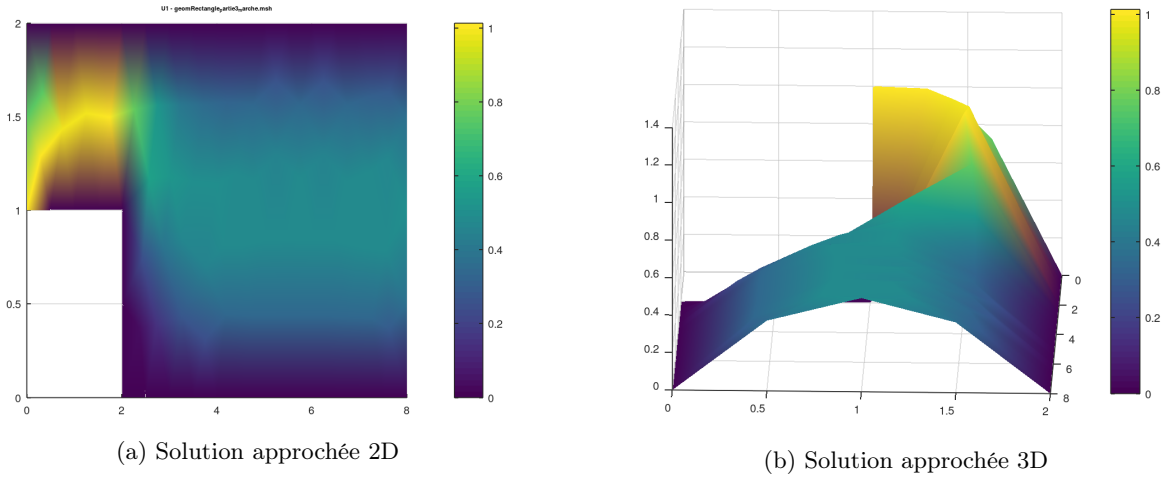


Figure 67: Résultats pour u_1 pour $\nu = 1$ et $h = 0.5$

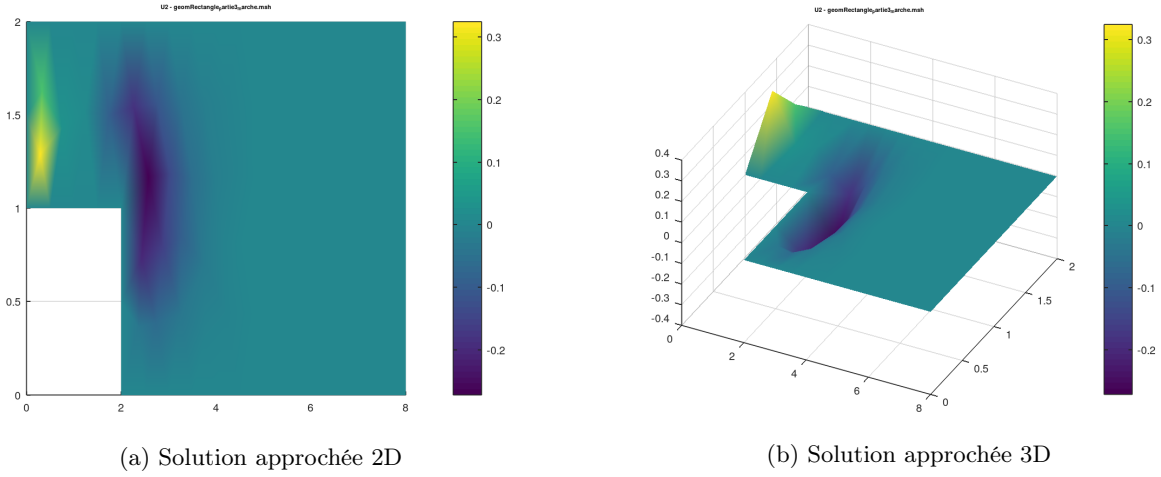


Figure 68: Résultats pour u_2 pour $\nu = 1$ et $h = 0.5$

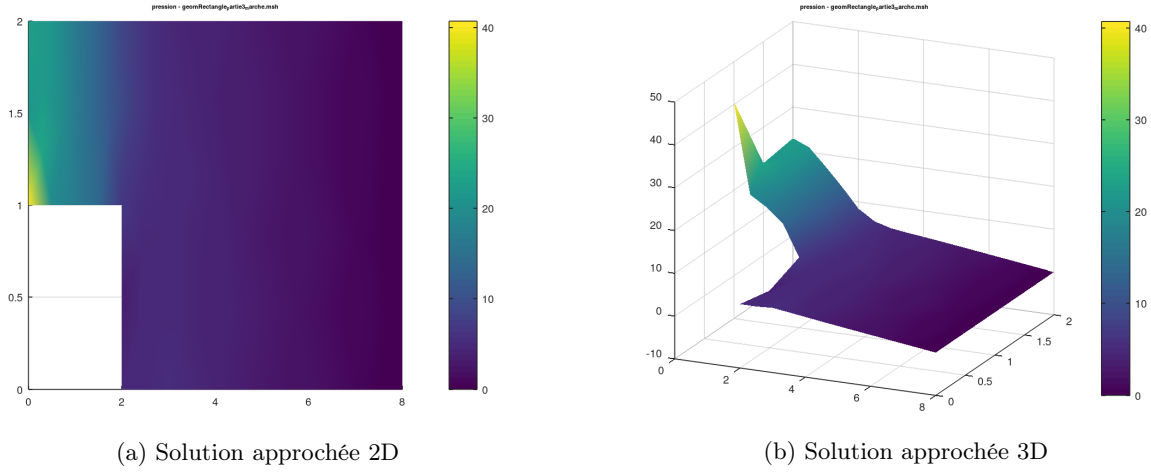


Figure 69: Résultats pour p pour $\nu = 1$ et $h = 0.5$

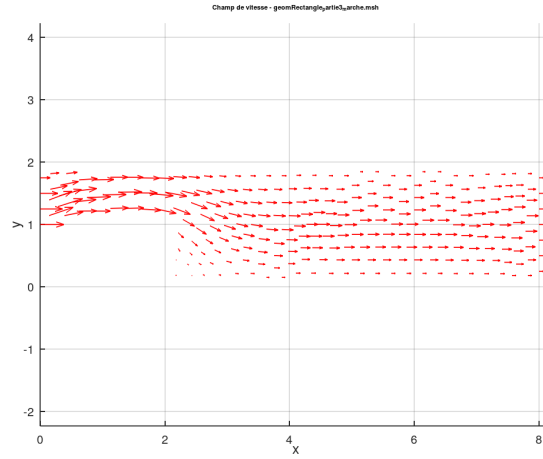


Figure 70: Vecteurs vitesse pour $\nu = 1$ et $h = 0.5$

On observe bien des résultats attendus pour l'écoulement d'un fluide dans un canal avec une marche descendante à savoir :

- **Pression (première figure, 3D) :**
 - La pression diminue significativement juste après la marche descendante, reflétant une accélération du fluide dans cette zone.
 - La transition vers une section élargie entraîne une perte de pression en aval de la marche.
 - Un gradient de pression est visible dans la direction longitudinale, avec une pression plus élevée à proximité immédiate de la marche.
- **Composante u_1 de la vitesse (deuxième figure, 3D) :**
 - Une variation importante de u_1 (composante longitudinale de la vitesse) est observée autour de la marche.
 - En aval de la marche, le fluide accélère progressivement, en accord avec la diminution de la pression.
 - Une discontinuité au niveau de la marche est causée par le changement brusque de la géométrie, perturbant le champ de vitesse.
- **Champ de vitesse (troisième figure, vecteur 2D) :**

- Le champ vectoriel montre une accélération claire du fluide en aval de la marche, avec une réorganisation des lignes de courant.
- Une éventuelle recirculation pourrait être présente juste derrière la marche, un phénomène typique dans ce type d'écoulement.
- En aval, l'écoulement semble se stabiliser avec des lignes de courant reprenant une structure parallèle.

4.8.4 Influence de la viscosité

Regardons pour différents ν qui caractérisent la viscosité du fluide. On trace les 3 graphiques 3D pour u_1 et p pour $h = 0.5$ et $\nu = 0.01, 1, 100$:

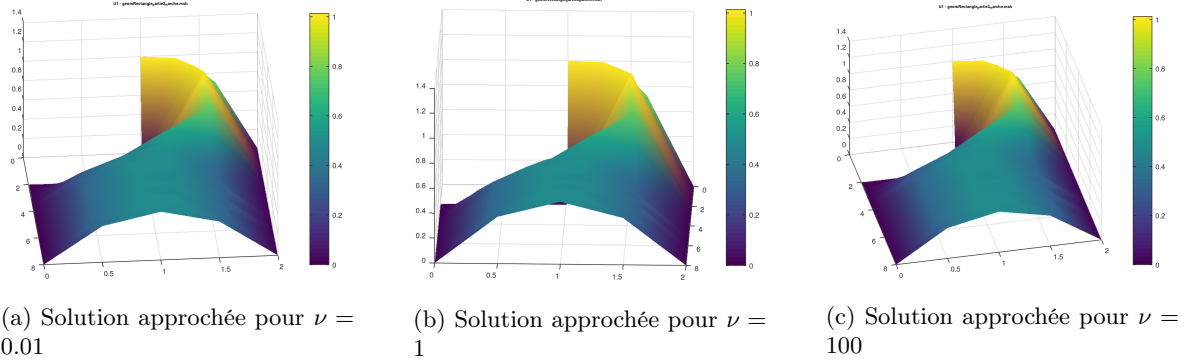


Figure 71: Résultats pour u_1 pour $h = 0.5$ et différents ν

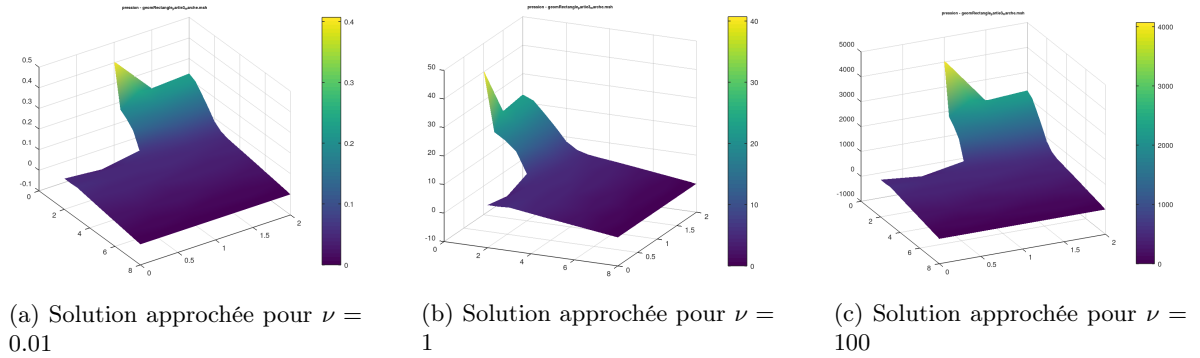


Figure 72: Résultats pour p pour $h = 0.5$ et différents ν

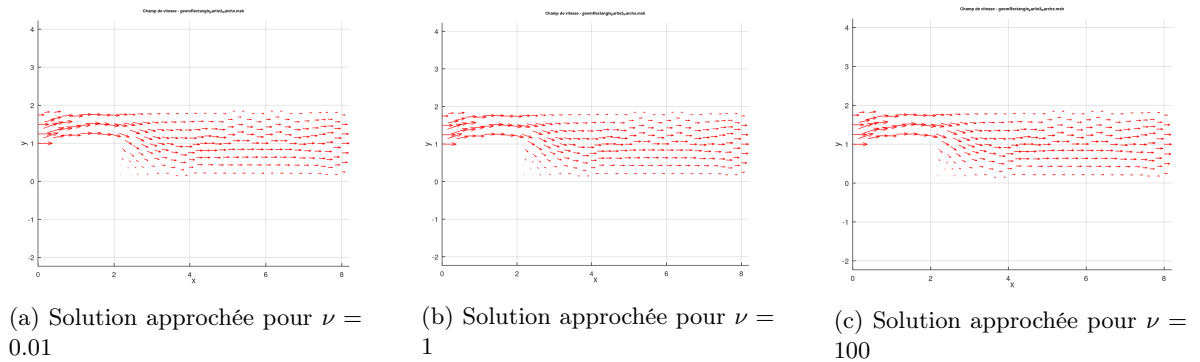


Figure 73: Vecteurs vitesse pour $h = 0.5$ et différents ν

On observe des tendances similaires pour les différentes valeurs de ν , on peut expliquer :

L'influence de la viscosité sur les comportements des solutions est fondamentale pour comprendre les dynamiques des fluides dans des géométries complexes, comme celle d'une marche descendante dans un canal. La viscosité détermine essentiellement la capacité du fluide à résister aux déformations dues à des forces externes, ce qui influence considérablement les caractéristiques de l'écoulement, telles que la pression, la vitesse, et la structure du champ de vitesse. Voici les principales variations observées en fonction de la viscosité :

- À faible viscosité, l'écoulement est généralement plus turbulent, avec des fluctuations importantes dans la vitesse et la pression. Ce type d'écoulement est plus sensible aux petites perturbations et génère souvent des structures complexes, comme des vortex. La pression sur le fluide dans la marche est faible.
- À haute viscosité, l'écoulement devient plus laminaire, plus stable, et moins sensible aux perturbations. La dissipation visqueuse devient dominante et "ralentit" le fluide, réduisant ainsi l'intensité des fluctuations. On voit dans les graphiques des vecteurs vitesses que la vitesse va à plus d'endroit de la marche, elle n'évite plus une partie de la marche comme avant. La pression sur le fluide dans la marche est plus élevée.

En résumé, la viscosité a un impact majeur sur la stabilité de l'écoulement et la distribution de la pression et de la vitesse dans un fluide. Une faible viscosité favorise des comportements plus dynamiques, mais aussi plus instables, tandis qu'une viscosité élevée tend à rendre l'écoulement plus prévisible et stable.